

西南交通大學
Southwest Jiaotong University

Operating System

Process Management

Guangyao Zhou

guangyao_zhou@swjtu.edu.cn

Content

1. Processor Management
2. Process（进程）
3. Thread（线程）
4. Scheduling （调度）
5. Concurrency (并发)
6. Synchronization and Deadlock



Content

1. **Processor Management**
2. **Process**
3. **Thread**
4. **Scheduling （调度）**
5. **Concurrency (并发)**
6. **Synchronization and Deadlock**



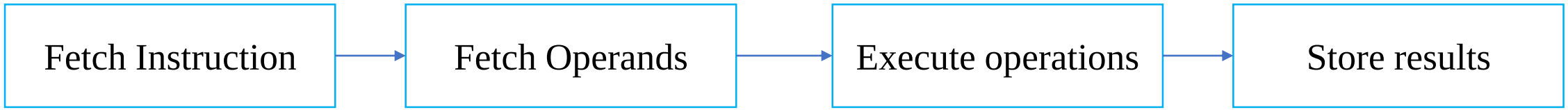
1. Processor Management

- Processor management: is an important component of the operating system, responsible for managing, scheduling, and allocating the critical resources of the computer system - processors - and controlling the execution of programs.
- The basic task of an operating system is to manage processes
  - Control process execution
  - Allocate resources to processes
  - Allow processes to share and exchange information
  - Protect each process from others during runtime
- In order to improve the granularity of concurrency （并发） and reduce the overhead of concurrency, modern operating systems have further introduced the concept of threads

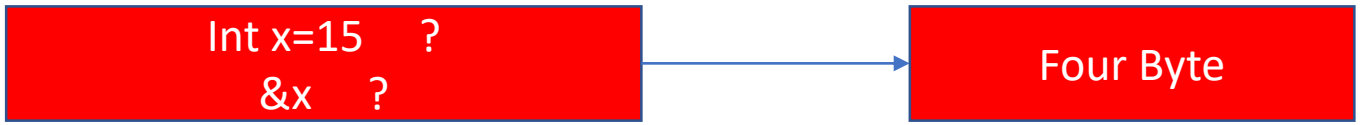
Central Processing Unit（中央处理器）

The basic function of a CPU is to implement correct control over the flow of instructions and data in time and space.

- **Instruction control: Generate the address of the next instruction in memory.**
- **Operation control: Generate various operation signals to send to corresponding components, to control the actions required to complete instructions.**
- **Timing control: Implementing temporal control over various operation signals, to ensure that the computer works continuously and automatically in an orderly manner.**



Instruction Execution



Composition of CPU

- **Control component**: Composed of program counter (PC), instruction register (IR), timing generator, and micro operation forming components
- **Arithmetic component**: Composed of an arithmetic logic unit (ALU，逻辑运算单元), an accumulation register, a data buffer register, and a state condition register
- **Register set**: General register, Special register (Program counter (PC), Instruction Register (IR), Memory Data Register (MDR), Memory Address Register (MAR), Program Status Word Register (PSW)……)
- **Internal data path of CPU**
- In modern computers, **cache** is also included

Program Execution

- **The basic function of a computer is to execute programs, and the ultimately executed programs are machine instructions (binary encoding) stored in memory.**
- **The processor retrieves an instruction from memory to the instruction register (IR) based on the program counter (PC) and executes it. The PC will automatically increment or change to a transition address to indicate the next instruction to be executed.**

i++ ⇒ MOV R1,#0
ADD R1,#1 or INC R1 ⇒ Binary signal ⇒ 01001→01010

Processor Status

- **Processor Status**: also known as **operating mode of the processor**. How does the central processing unit know whether it is currently running an operating system or a regular user on it? This will depend on the flags of the processor state. Set the **processor to different states based on the usage permissions of resources and machine instructions** during the execution of different programs.
- **Management Status**: Processor can execute all instructions, access all resources and have the ability to change processor status.
- **User Status**: Processor can only execute non privileged instructions.

Program Status Word and Interrupt

- **Program Status Word (PSW):** How does a computer know what working state it is currently in? Can privileged instructions be executed at this time? Usually, operating systems introduce the PSW to distinguish the working status of different processors.
- Interruption (中断): refers to the interruption of the current program running on the CPU during the execution of a program when an event occurs, introduce the process of executing the service program that handles the event.

Content

1. Processor Management
2. **Process**
3. Thread
4. Scheduling （调度）
5. Concurrency (并发)
6. Synchronization and Deadlock



2. Process Description and Control

- **Process concept**
- Process description
- Process execution

Program Example

Sum of all the numbers of a dataset (S):

Single Thread

```
A = 0
for i in S:
    A += i
```

Multiple Threads

```
A1, A2, …, An = 0
Split S into multiple set (S1, S2, …, Sn)
for i =1 to n:
    initial and begin thread_i to obtain the sum of Si
threads join()
Sum A1, A2,…, An
```

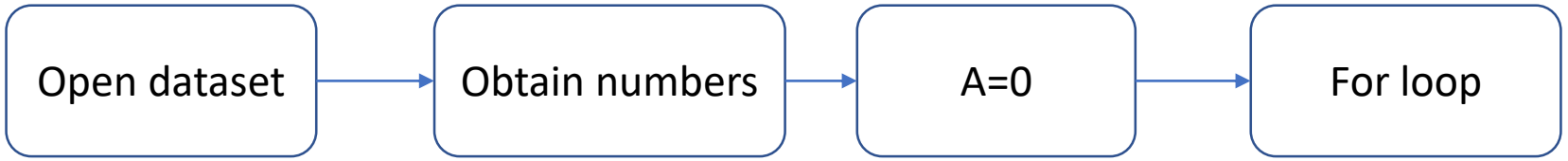
Program Example

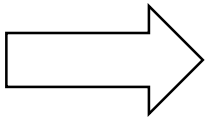
Sum of all the numbers of a dataset (S):

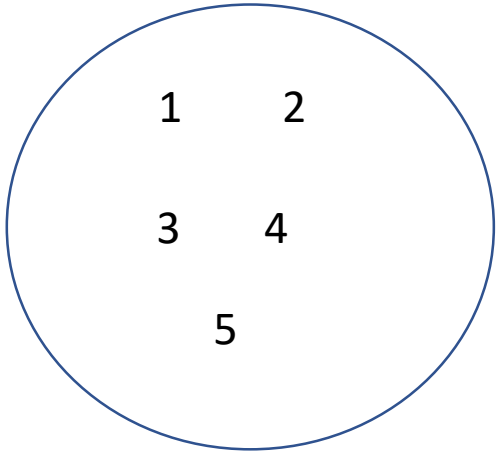
Single Thread

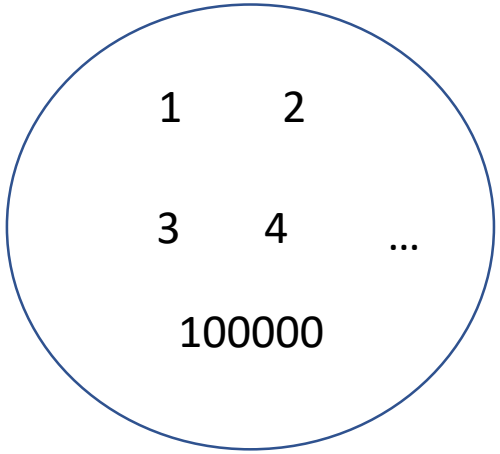
```
A = 0
for i in S:
    A += i
```

Sequential execution: Subsequent operations can only be executed after the current operation (program segment) is completed







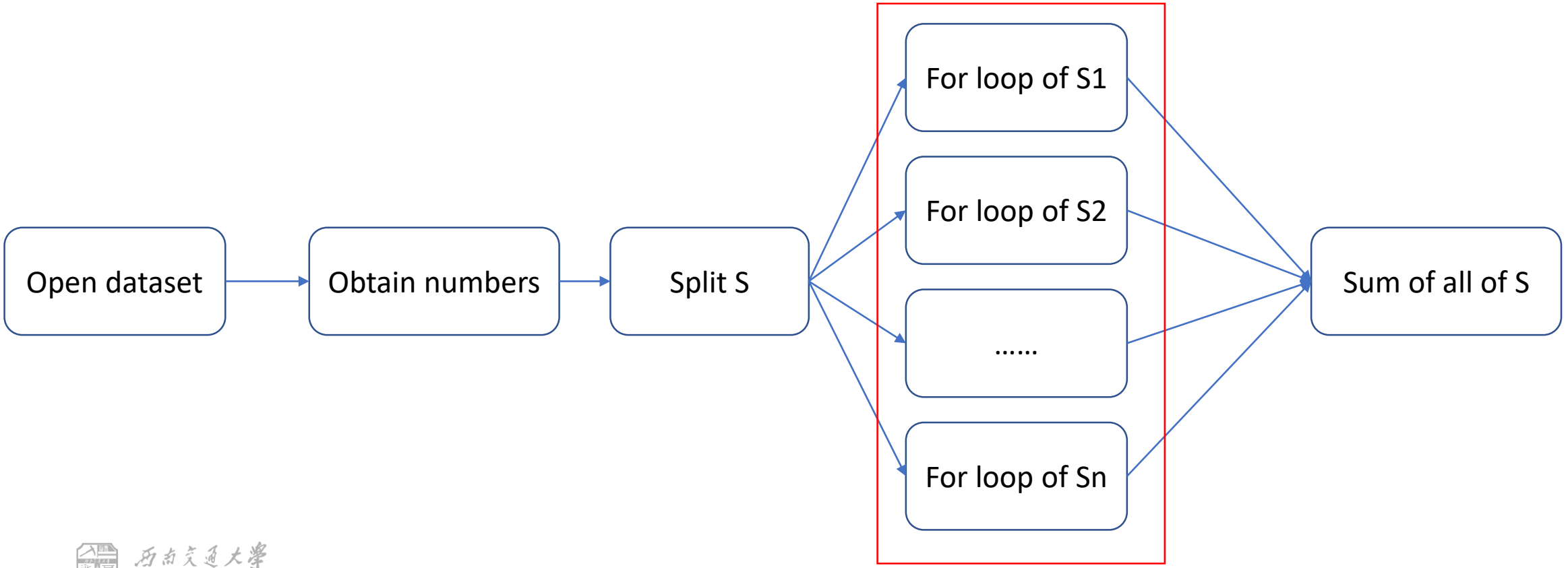


Different data can lead to different results with the same program

Program Example

Sum of all the numbers of a dataset (S):

Multiple Threads: multiple threads can run simultaneously (parallel or concurrent)



Program Example

Ping all the IPs of local net:

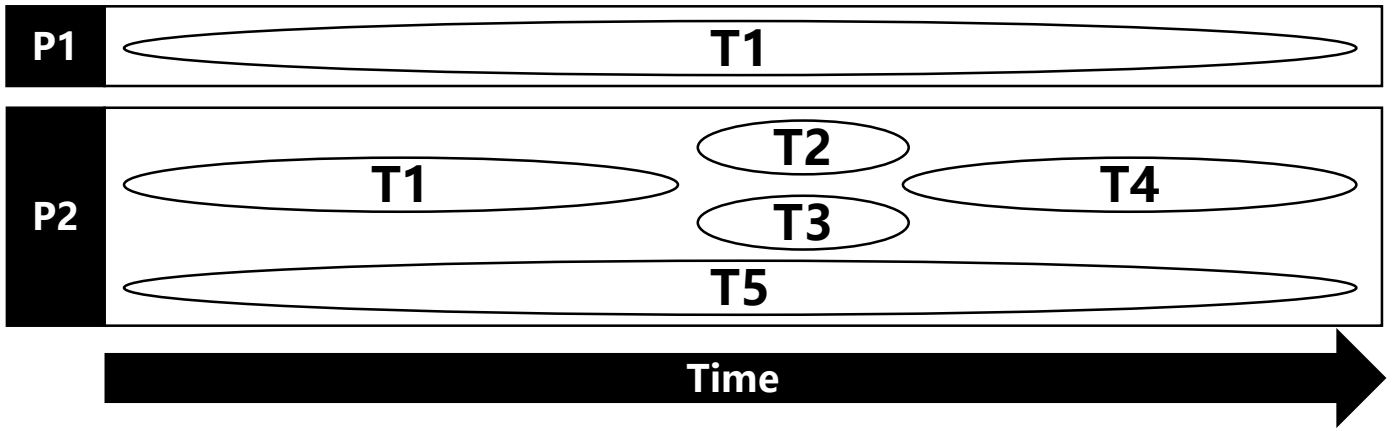
```
for /L %i in (1,1,255) do ping -n 1 -w 100 192.168.1.%i

for /L %i in (1,1,255) do ping -n 1 -w 100 192.168.1.%i | find "TTL="
```

Process and Thread

- **Process 进程**
  - **The definition of a process: A process is one-time execution process of a concurrently executable program with independent functionality regarding a certain dataset, also the basic unit for resource allocation and protection in operating systems. (1978 National Conference on Operating Systems)**

- **Thread 线程**
  - **The definition of a thread: A thread is an execution unit (a single sequential control flow) in a process, responsible for the execution of programs in the current process. There is at least one thread in a process.**



Process Concept

Process 进程

- **Definition of a process: A process is one-time execution process of a concurrently executable program with independent functionality regarding a certain dataset, also the basic unit for resource allocation and protection in operating systems. (1978 National Conference on Operating Systems)**
- **A process is an instance of an executing program**
- **Origin**
  - **It is a concept introduced after the emergence of multiprogramming systems to depict the dynamic situation within the system and describe the activity patterns of each program within the system. All operating systems designed with multiprocessing are based on processes.**
  - **1960: MIT—process, IBM—task, Univac—active**

Process Concept

- Resources are made available to multiple applications
- The processor is switched among multiple applications so all will appear to be progressing
- The processor and I/O devices can be used efficiently
- An operating system executes a variety of programs:
  - Batch system – jobs
  - Time-shared systems – user programs or tasks

- Textbook uses the terms job and process almost interchangeably

Process Properties

- **Definition of a process: A process is one-time execution process of a concurrently executable program with independent functionality regarding a certain dataset, also the basic unit for resource allocation and protection in operating systems. (1978 National Conference on Operating Systems)**



- **Concurrency :** Processes can execute concurrently, and the concurrency performance of processes improves resource utilization and system efficiency (multiple processes take turns occupying the processor).
- **Independence:** A process is the basic unit for resource allocation and protection in a system - different processes do not share memory addresses.
- **Structurality:** A process consists of a data set and the programs running on it, including: program blocks, data blocks and process control blocks.

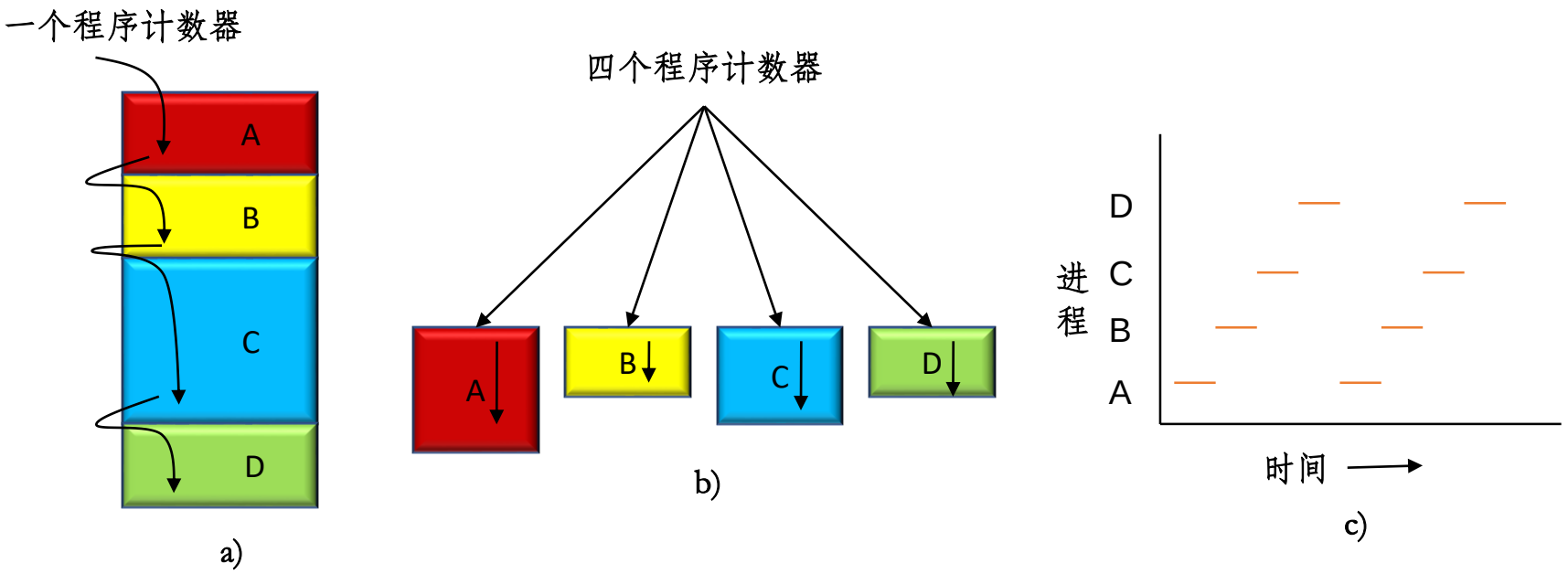
Process Properties

- **Definition of a process: A process is one-time execution process of a concurrently executable program with independent functionality regarding a certain dataset, also the basic unit for resource allocation and protection in operating systems. (1978 National Conference on Operating Systems)**



- **Dynamics:** A process is the execution of a program on a set of data. It is a dynamic concept with a life cycle.
- **Restrictiveness (Relevance):** There is a constraint relationship among processes: Different processes may need to wait for or communicate with each other.
- **Shareability:** The same program can run on different datasets simultaneously, forming different processes. (Sharing of programs, not processes)

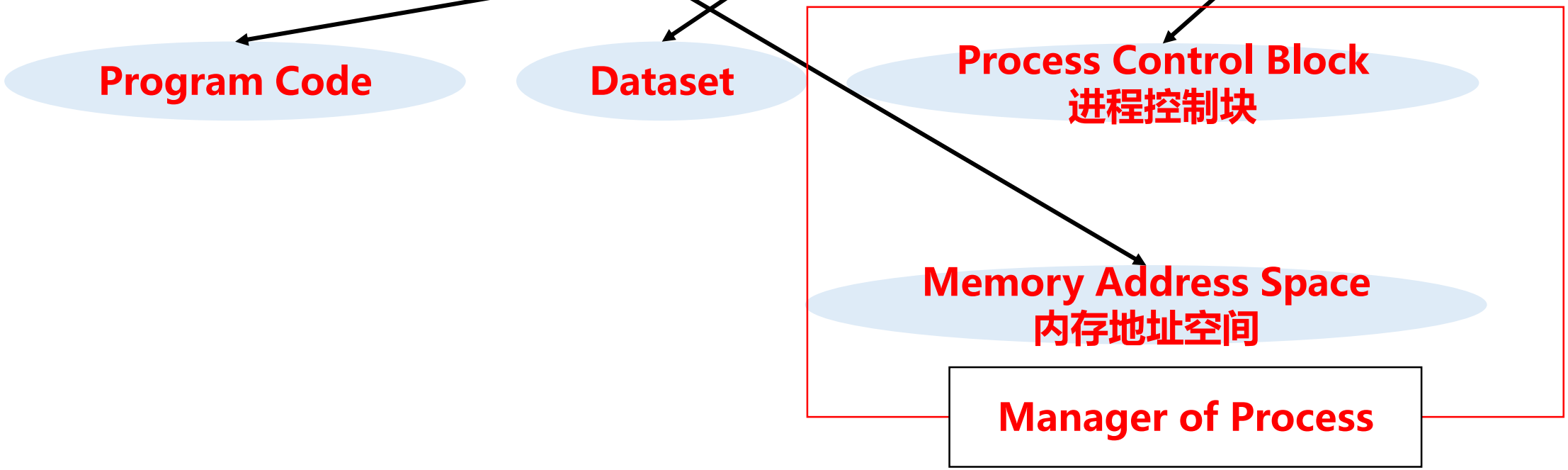
Example of Process Model and Independence



In single processor: a）含有4道程序的多道程序；b）4个独立顺序进程的概念模型；c）在任一时刻仅有一个程序是活跃的

Process Elements

- **Definition of a process: A process is one-time execution process of a concurrently executable program with independent functionality regarding a certain dataset, also the basic unit for resource allocation and protection in operating systems. (1978 National Conference on Operating Systems)**



Process Elements

- Program code
  - which may be shared with other processes that are executing the same program
- A set of data associated with that code
  - When the processor begins to execute the program code, we refer to this executing entity as a process
- While the program is executing, this process can be uniquely characterized by a number of elements, including:
  - identifier
  - state
  - priority
  - program counter
  - memory pointers
  - context data
  - I/O status information
  - accounting information

Process Control Block（进程控制块）

- Contains the process elements
- It is possible to interrupt a running process and later resume execution as if the interruption had not occurred
- Created and managed by the operating system
- Key tool that allows support for multiple processes
- Role: to make a program (including data) that cannot run independently in a multiprogramming environment a basic unit that can run independently and a process that can execute concurrently with other processes. Or in other words, the OS controls and manages concurrent processes based on the PCB.

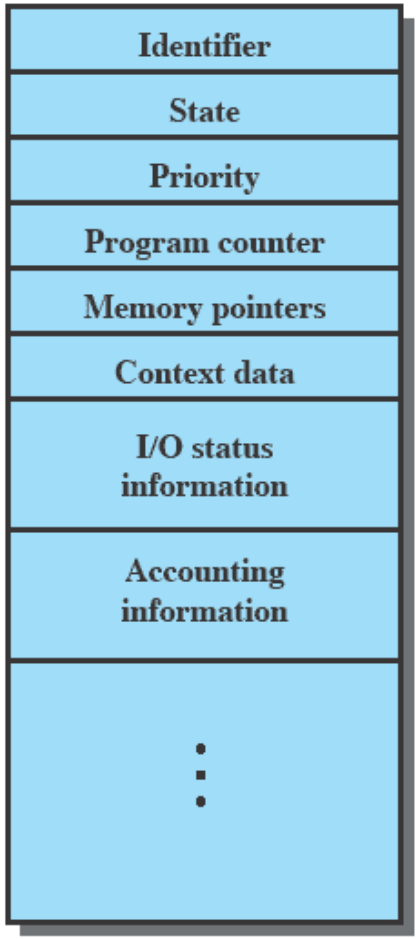
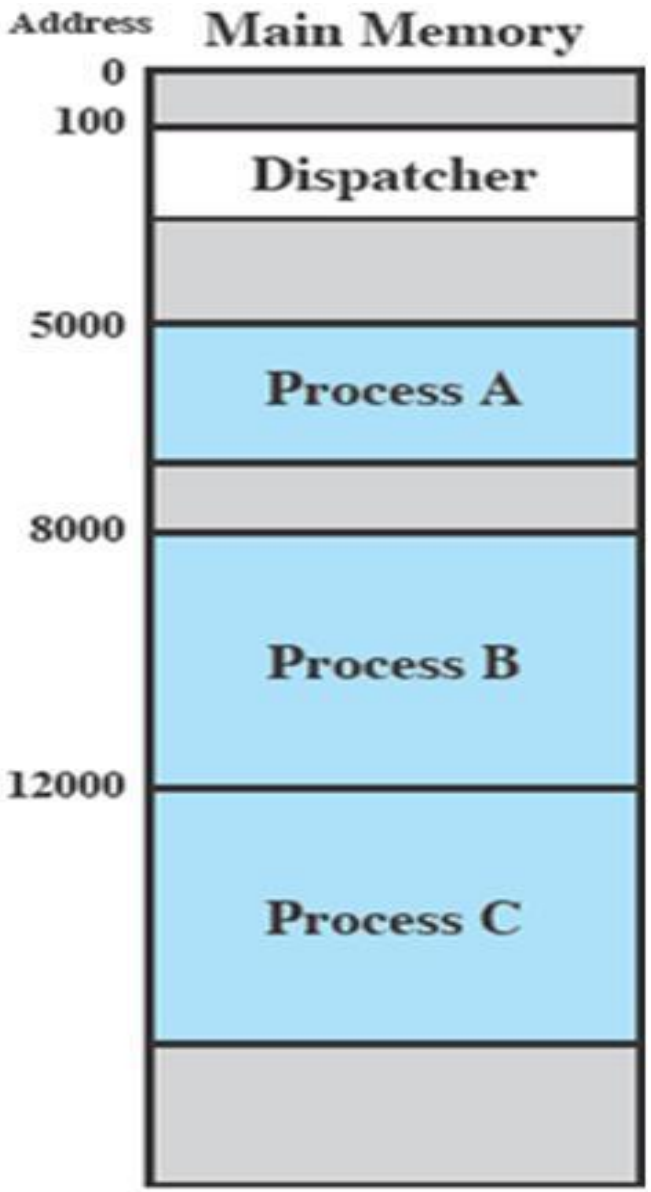


Figure 3.1 Simplified Process Control Block

Process States

- Trace
  - the behavior of an individual process by listing the sequence of instructions that execute for that process
  - the behavior of the processor can be characterized by showing how the traces of the various processes are interleaved
- Dispatcher
  - small program that switches the processor from one process to another



Traces of Processes

(a) Trace of Process A	(b) Trace of Process B	(c) Trace of Process C
5000	8000	12000
5001	8001	12001
5002	8002	12002
5003	8003	12003
5004		12004
5005		12005
5006		12006
5007		12007
5008		12008
5009		12009
5010		12010
5011		12011

5000 = Starting address of program of Process A
8000 = Starting address of program of Process B
12000 = Starting address of program of Process C

Figure 3.3 Traces of Processes of Figure 3.2

西南交通大學 Southwest Jiaotong University

Combined Trace of Processes

1	5000
2	5001
3	5002
4	5003
5	5004
6	5005
----------	Timeout
7	100
8	101
9	102
10	103
11	104
12	105
13	8000
14	8001
15	8002
16	8003
----------	I/O Request
17	100
18	101
19	102
20	103
21	104
22	105
23	12000
24	12001
25	12002
26	12003
27	12004
28	12005
----------	Timeout
29	100
30	101
31	102
32	103
33	104
34	105
35	5006
36	5007
37	5008
38	5009
39	5010
40	5011
----------	Timeout
41	100
42	101
43	102
44	103
45	104
46	105
47	12006
48	12007
49	12008
50	12009
51	12010
52	12011
----------	Timeout



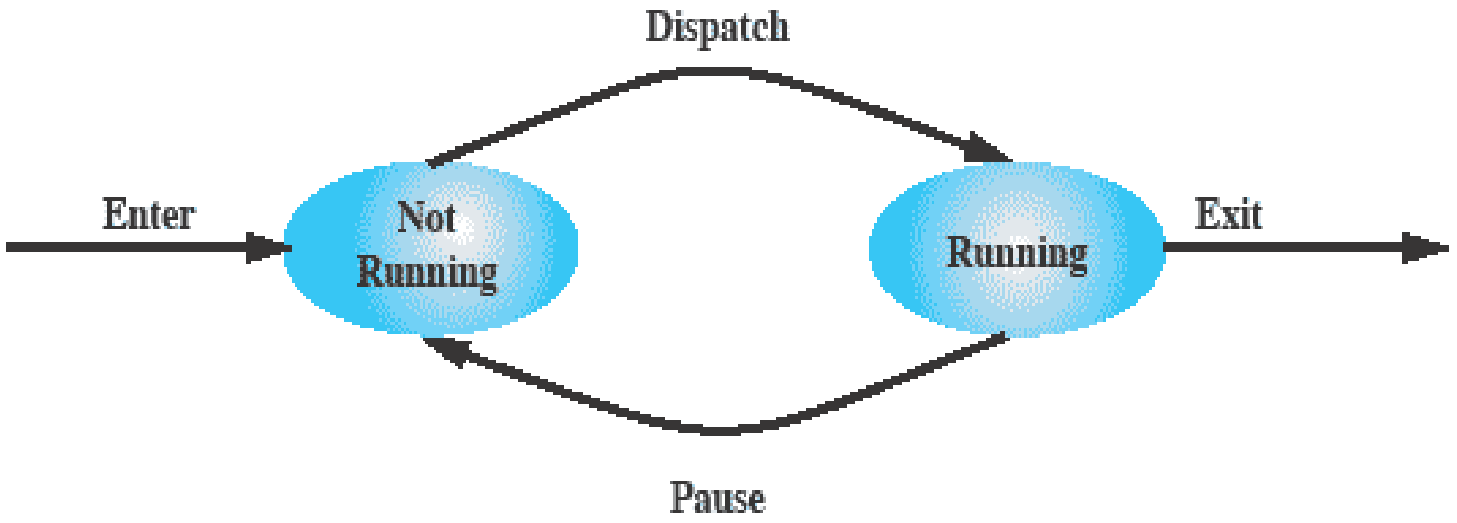
100 = Starting address of dispatcher program

Shaded areas indicate execution of dispatcher process;
first and third columns count instruction cycles;
second and fourth columns show address of instruction being executed

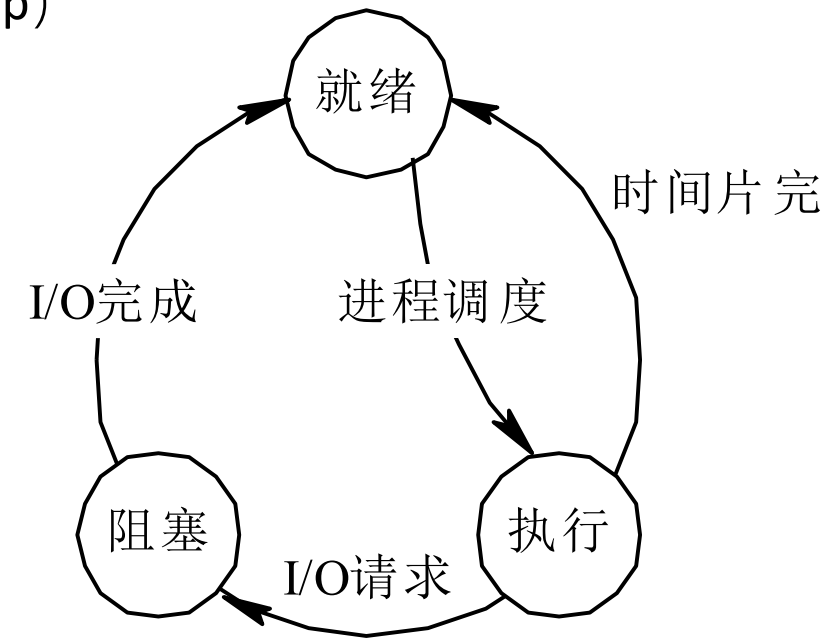
Figure 3.4 Combined Trace of Processes of Figure 3.2

Two-State Process Model

- A process may be in one of two states:
  - running
  - not-running: ready（就绪）, blocked（阻塞, or wait, or sleep）

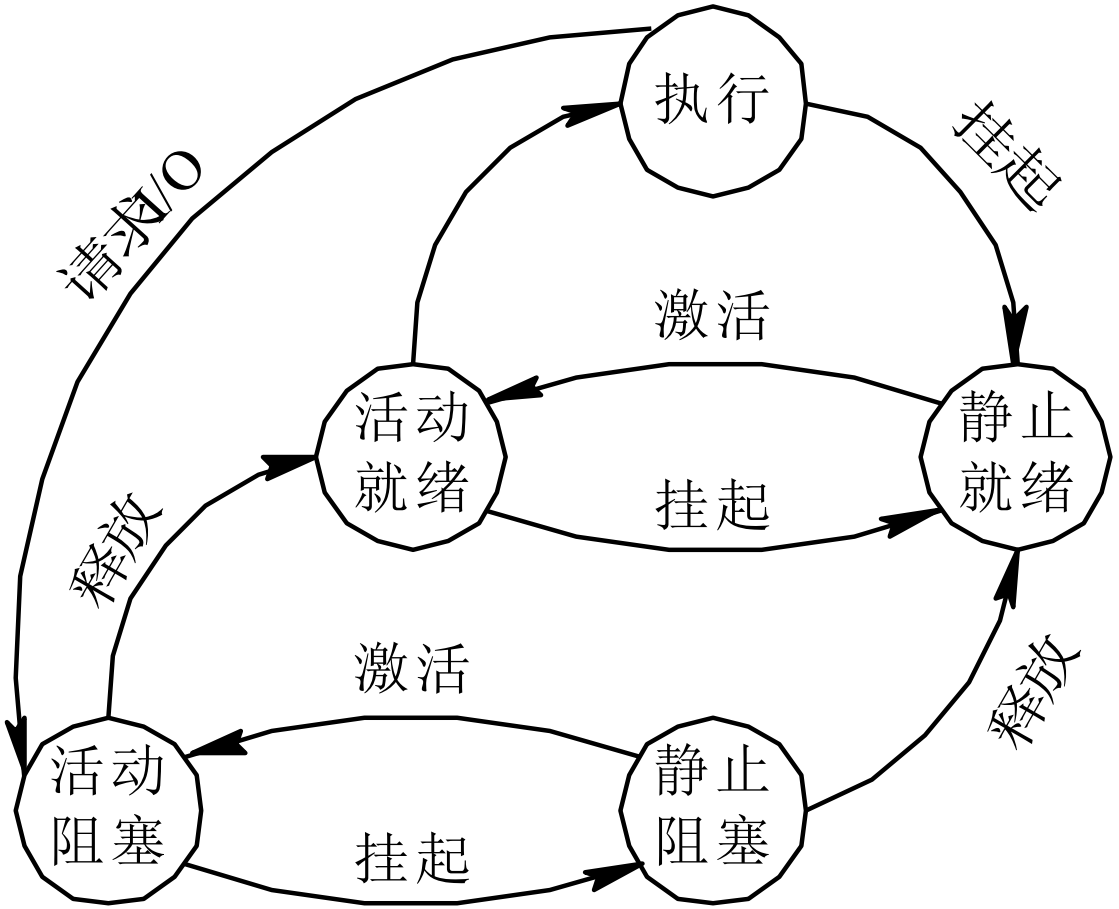


(a) State transition diagram



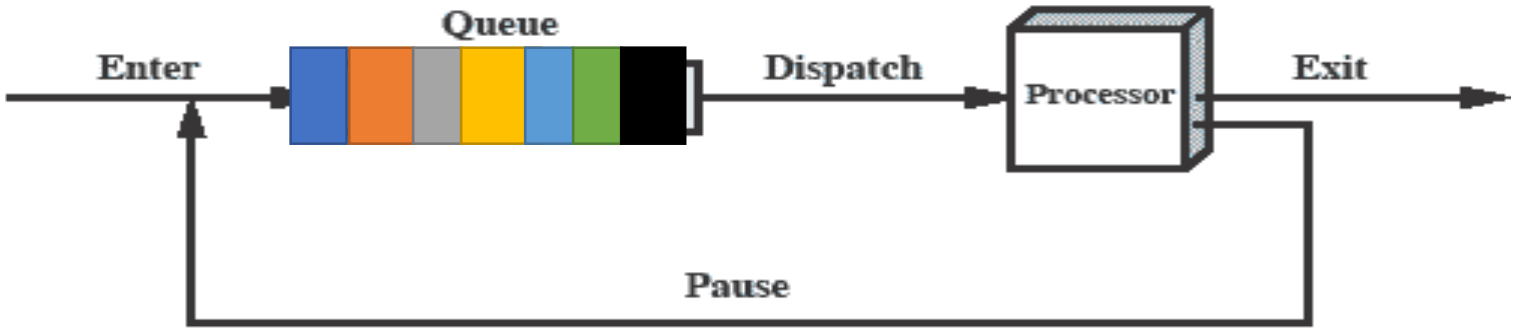
(b) The Three Basic States of Processes and Their Transitions

Process State Graph



Process state graph with suspended state

Queuing Diagram



(b) Queuing diagram

Reasons for Process Creation

- UNIX: fork
- WINDOWS: CreateProcess

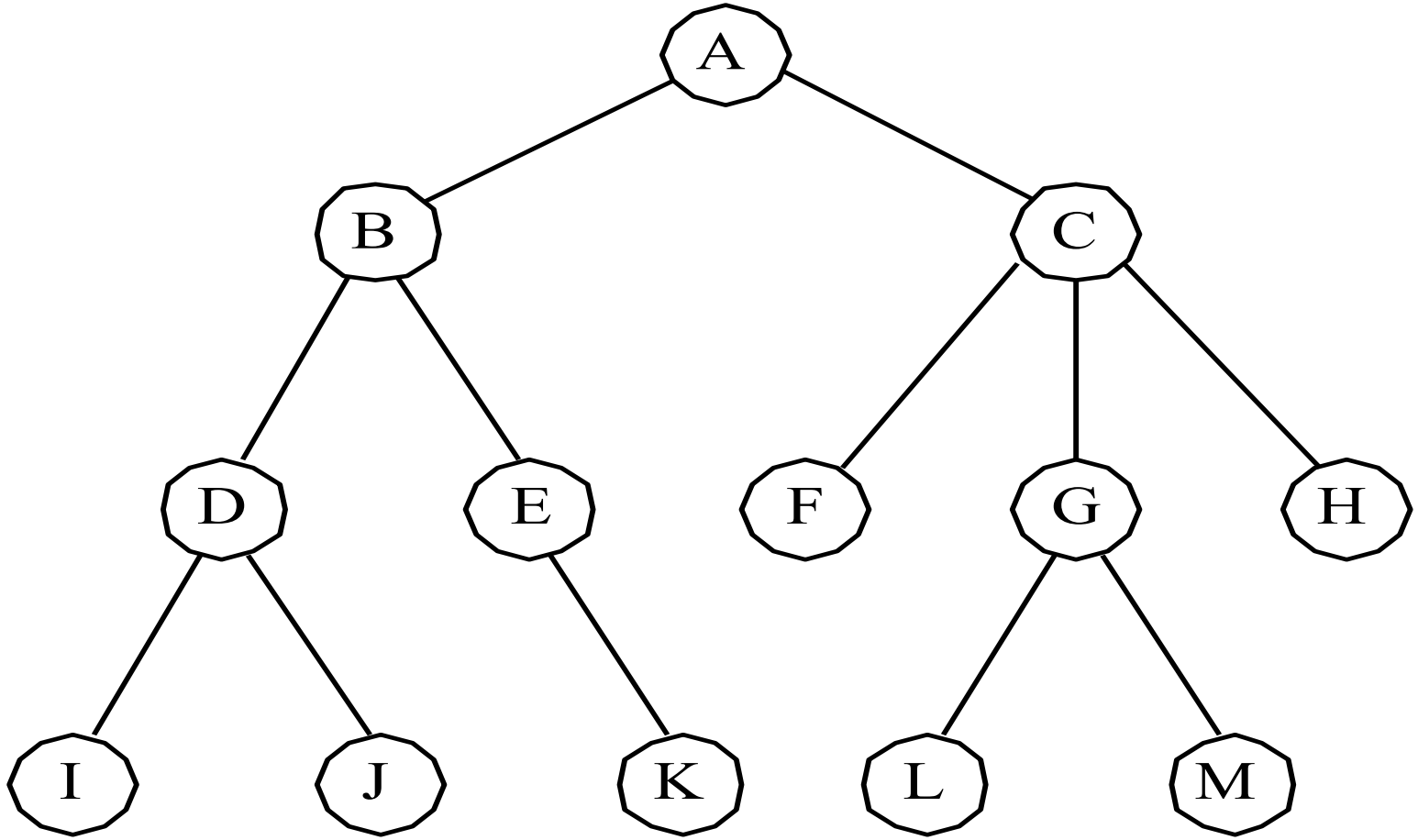
New batch job	The OS is provided with a batch job control stream, usually on tape or disk. When the OS is prepared to take on new work, it will read the next sequence of job control commands.
Interactive logon	A user at a terminal logs on to the system.
Created by OS to provide a service	The OS can create a process to perform a function on behalf of a user program, without the user having to wait (e.g., a process to control printing).
Spawned by existing process	For purposes of modularity or to exploit parallelism, a user program can dictate the creation of a number of processes.

Parent and child processes have their own distinct address spaces

Process Creation

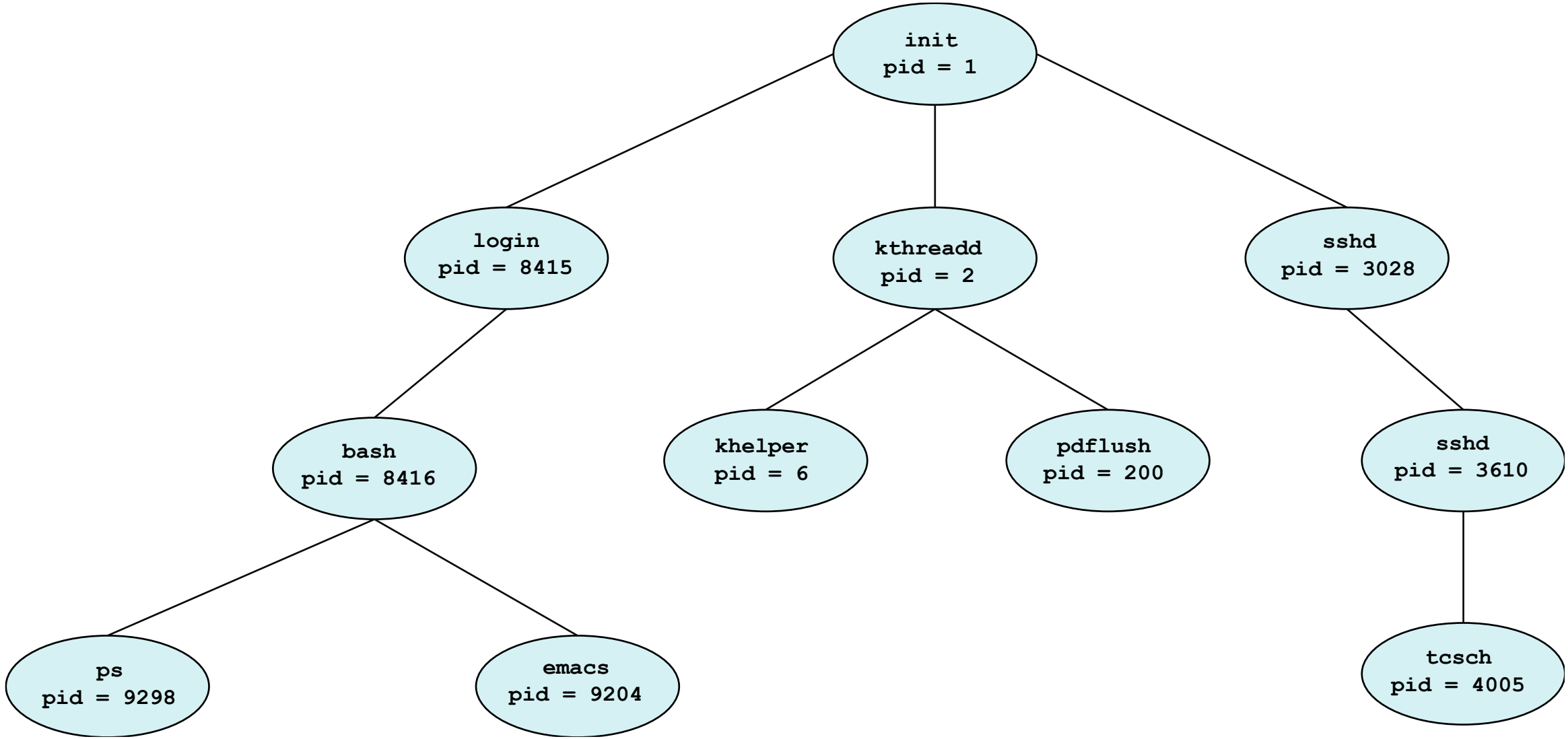
- **Process spawning**
  - when the OS creates a process at the explicit request of another process
- **Parent process**
  - is the original, creating, process
- **Child process**
  - is the new process

Process Graph



Process Tree

Tree of Processes in Linux



西南交通大學
Southwest Jiaotong University

Process Termination

- There must be a means for a process to indicate its completion
- A batch job should include a HALT instruction （停机指令） or an explicit OS service call for termination
- For an interactive application, the action of the user will indicate when the process is completed (e.g. log off, quitting an application)

- Conditions for Process Termination
  - Normal Termination (voluntary)
  - Error Termination (voluntary)
  - Serious error (involuntary)
  - Killed by other processes (involuntarily)

Process Termination

Normal completion	The process executes an OS service call to indicate that it has completed running.
Time limit exceeded	The process has run longer than the specified total time limit. There are a number of possibilities for the type of time that is measured. These include total elapsed time ("wall clock time"), amount of time spent executing, and, in the case of an interactive process, the amount of time since the user last provided any input.
Memory unavailable	The process requires more memory than the system can provide.
Bounds violation	The process tries to access a memory location that it is not allowed to access.
Protection error	The process attempts to use a resource such as a file that it is not allowed to use, or it tries to use it in an improper fashion, such as writing to a read-only file.
Arithmetic error	The process tries a prohibited computation, such as division by zero, or tries to store numbers larger than the hardware can accommodate.
Time overrun	The process has waited longer than a specified maximum for a certain event to occur.
I/O failure	An error occurs during input or output, such as inability to find a file, failure to read or write after a specified maximum number of tries (when, for example, a defective area is encountered on a tape), or invalid operation (such as reading from the line printer).
Invalid instruction	The process attempts to execute a nonexistent instruction (often a result of branching into a data area and attempting to execute the data).
Privileged instruction	The process attempts to use an instruction reserved for the operating system.
Data misuse	A piece of data is of the wrong type or is not initialized.
Operator or OS intervention	For some reason, the operator or the operating system has terminated the process (e.g., if a deadlock exists).
Parent termination	When a parent terminates, the operating system may automatically terminate all of the offspring of that parent.
Parent request	A parent process typically has the authority to terminate any of its offspring.

Combined Trace of Processes

?

1	5000	27	12004
2	5001	28	12005
3	5002	---------- Timeout	
4	5003	29	100
5	5004	30	101
6	5005	31	102
---------- Timeout		32	103
7	100	33	104
8	101	34	105
9	102	35	5006
10	103	36	5007
11	104	37	5008
12	105	38	5009
13	8000	39	5010
14	8001	40	5011
15	8002	---------- Timeout	
16	8003	41	100
---------- I/O Request		42	101
17	100	43	102
18	101	44	103
19	102	45	104
20	103	46	105
21	104	47	12006
22	105	48	12007
23	12000	49	12008
24	12001	50	12009
25	12002	51	12010
26	12003	52	12011
		---------- Timeout	

100 = Starting address of dispatcher program

Shaded areas indicate execution of dispatcher process;
first and third columns count instruction cycles;
second and fourth columns show address of instruction being executed

Figure 3.4 Combined Trace of Processes of Figure 3.2

Five-State Process Model

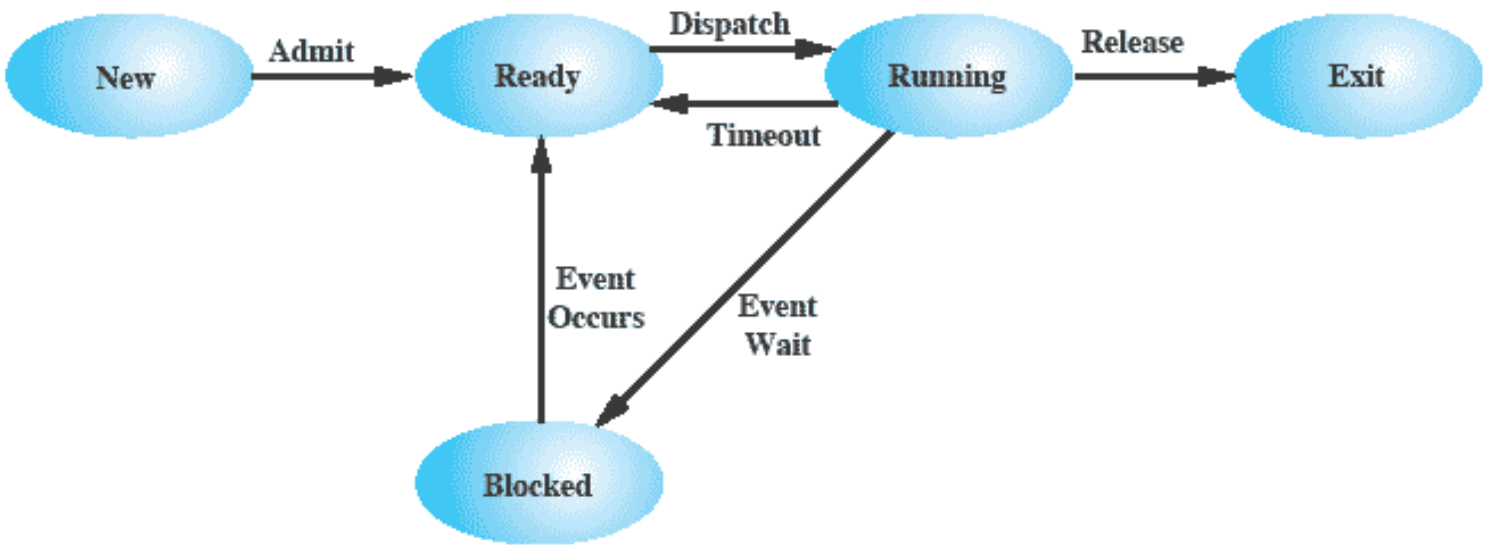
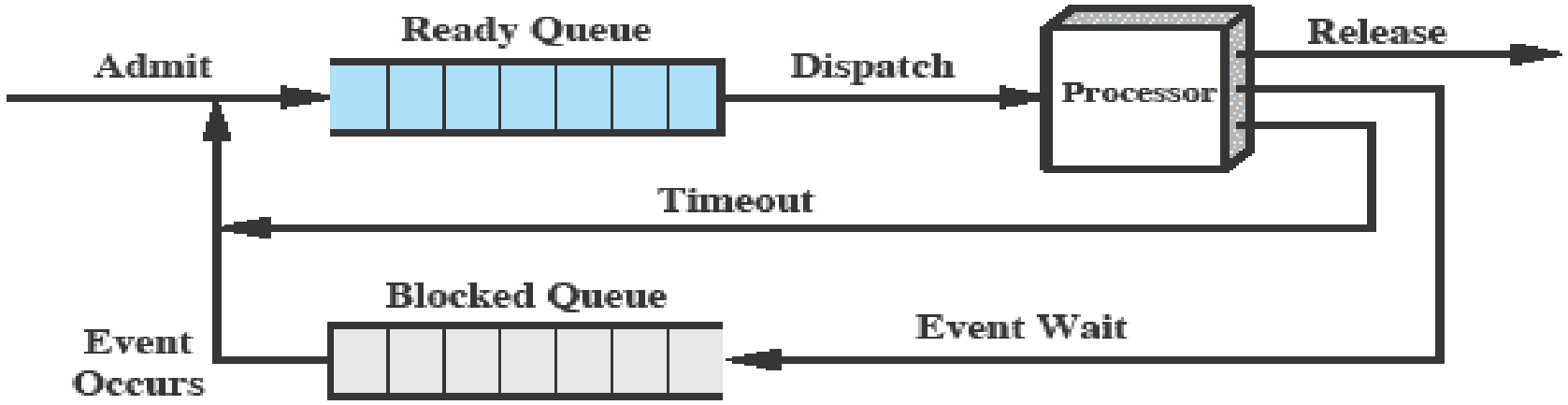


Figure 3.6 Five-State Process Model

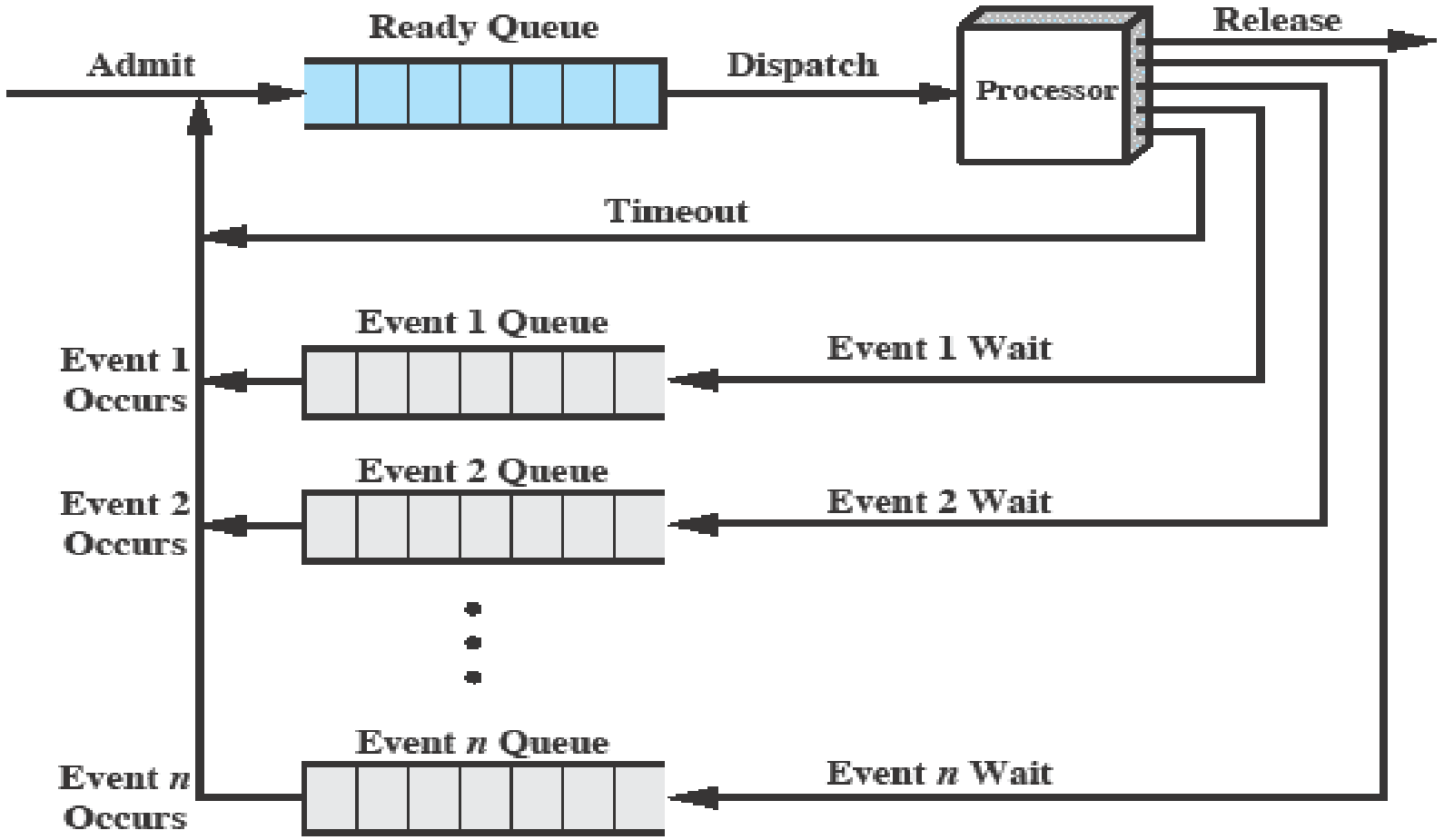
Using Two Queues

Process Queues: The data structure that links all process control blocks in the same state (such as ready state) together is called process queues, abbreviated as queues



(a) Single blocked queue

Multiple Blocked Queues

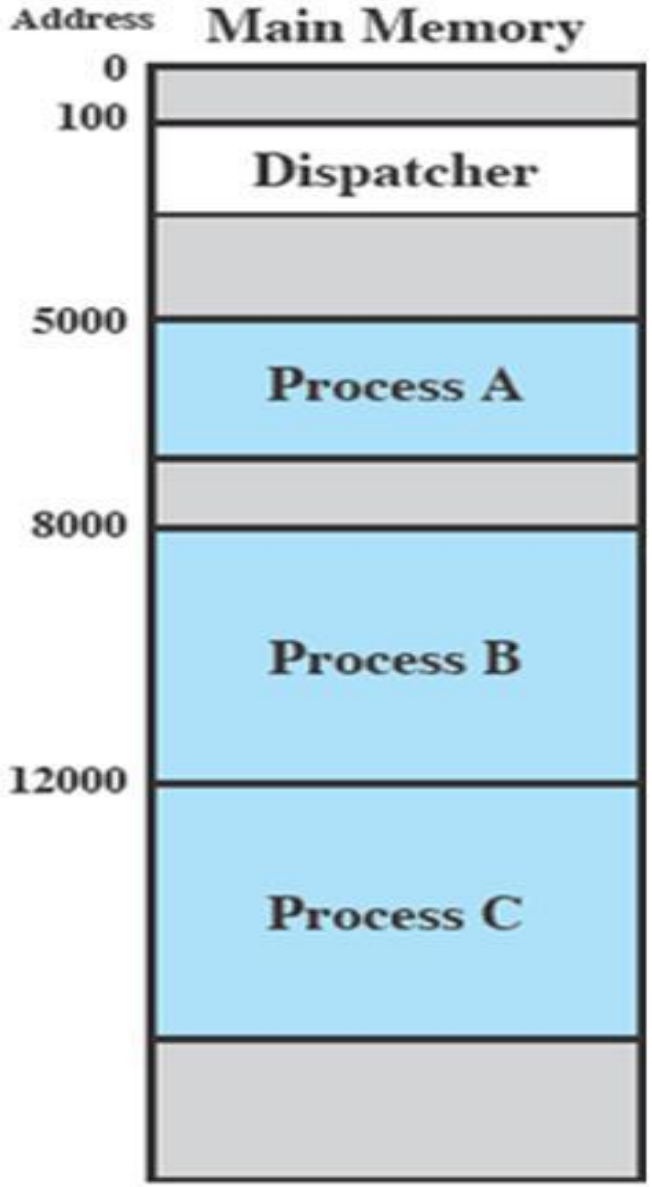


(b) Multiple blocked queues

西南交通大学
Southwest Jiaotong University

Suspended Processes

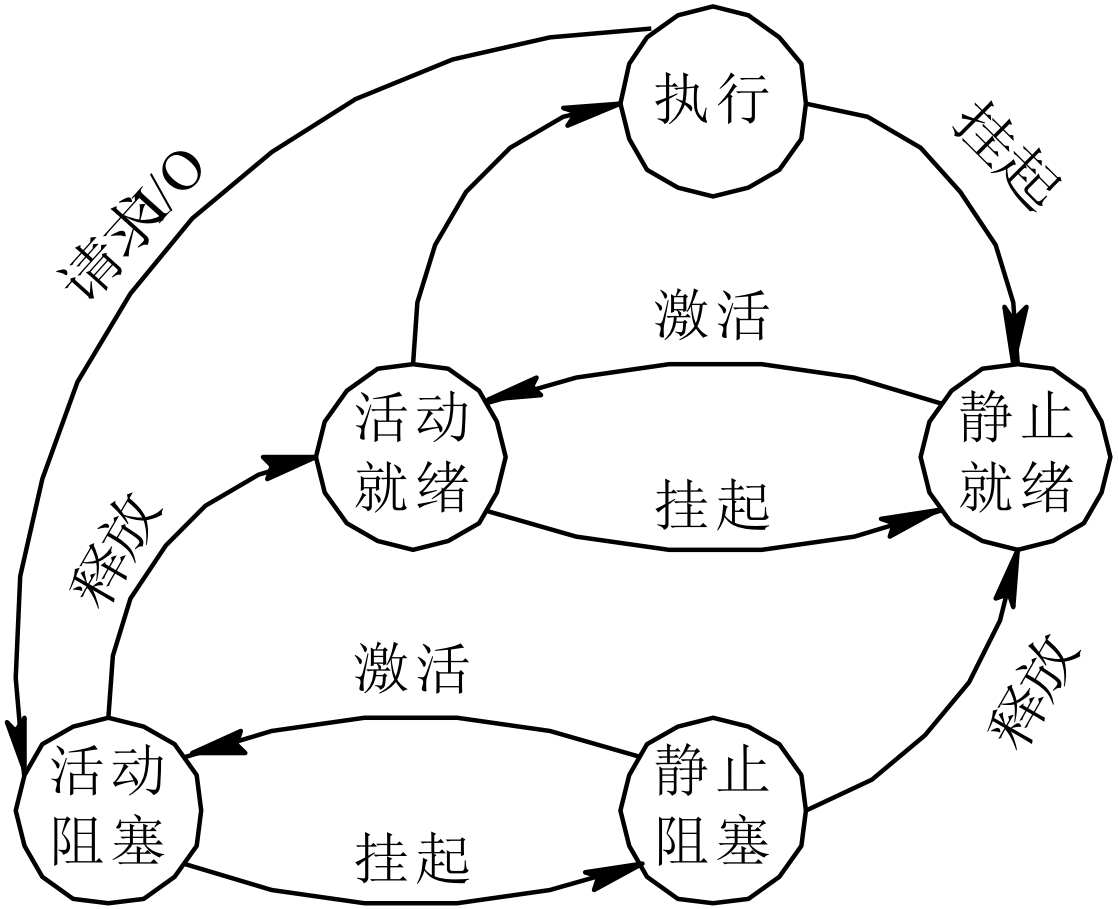
- Swapping (交换)
- involves moving part of all of a process from main memory to disk
- when none of the processes in main memory is in the Ready state, the OS swaps one of the blocked processes out on to disk into a suspend queue



Reasons for Process Suspension

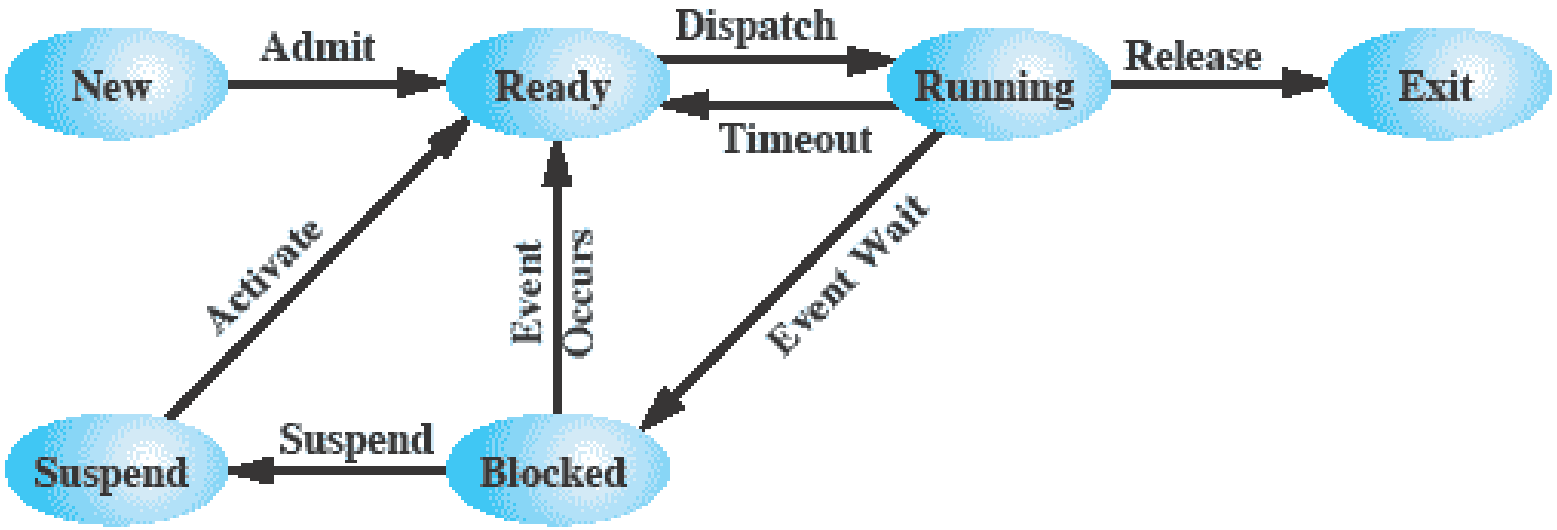
Swapping	The OS needs to release sufficient main memory to bring in a process that is ready to execute.
Other OS reason	The OS may suspend a background or utility process or a process that is suspected of causing a problem.
Interactive user request	A user may wish to suspend execution of a program for purposes of debugging or in connection with the use of a resource.
Timing	A process may be executed periodically (e.g., an accounting or system monitoring process) and may be suspended while waiting for the next time interval.
Parent process request	A parent process may wish to suspend execution of a descendent to examine or modify the suspended process, or to coordinate the activity of various descendants.

Process State Graph



Process state graph with suspended state

One Suspend State



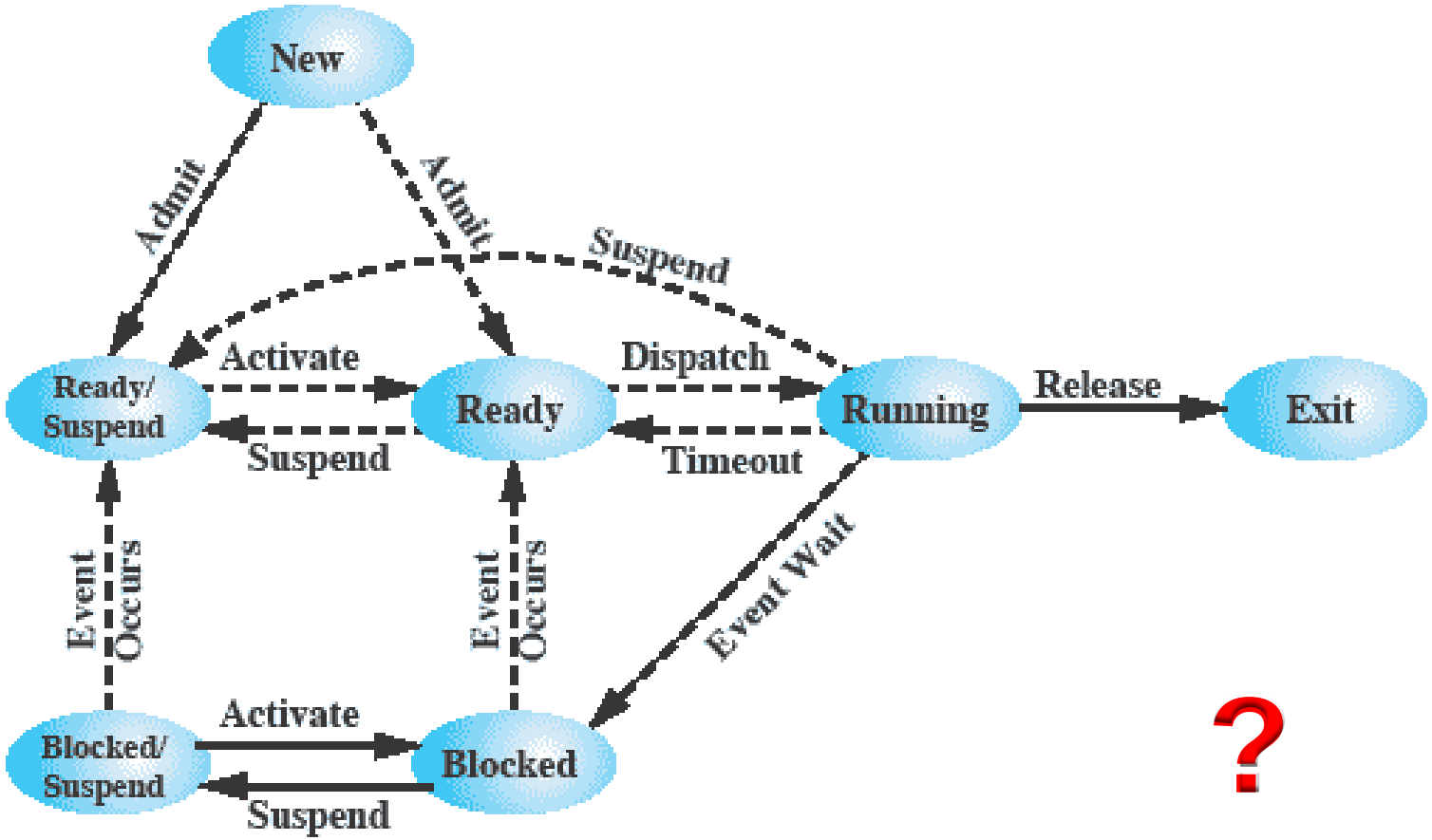
(a) With One Suspend State

?

Characteristics of a Suspended Process

- The process is not immediately available for execution
- The process was placed in a suspended state by an agent: either itself, a parent process, or the OS, for the purpose of preventing its execution
- The process may or may not be waiting on an event
- The process may not be removed from this state until the agent explicitly orders the removal

Two Suspend States



(b) With Two Suspend States

UNIX Process State

?

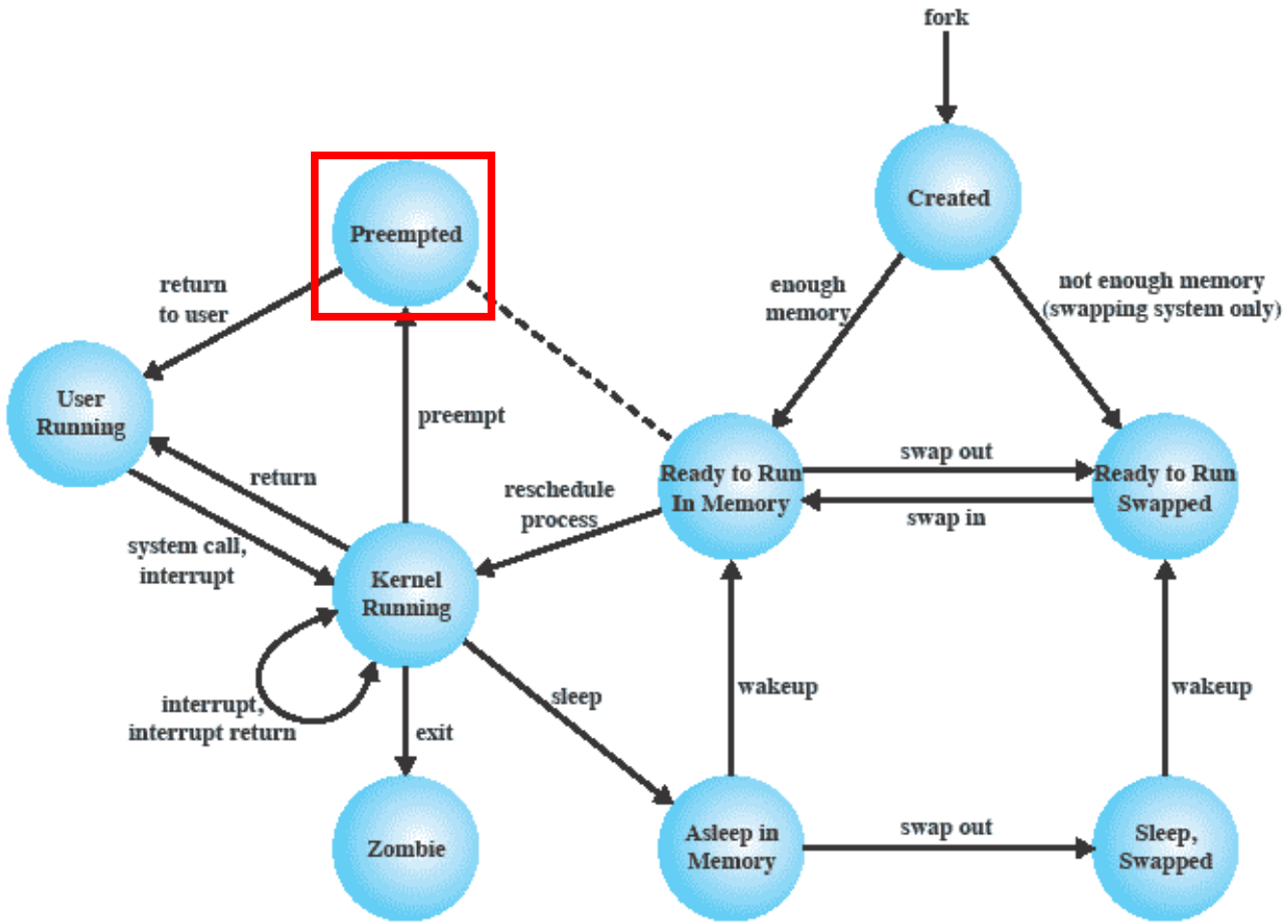


Figure 3.17 UNIX Process State Transition Diagram

2. Process Description and Control

- Process concept
- **Process description**
- Process execution

Process Description

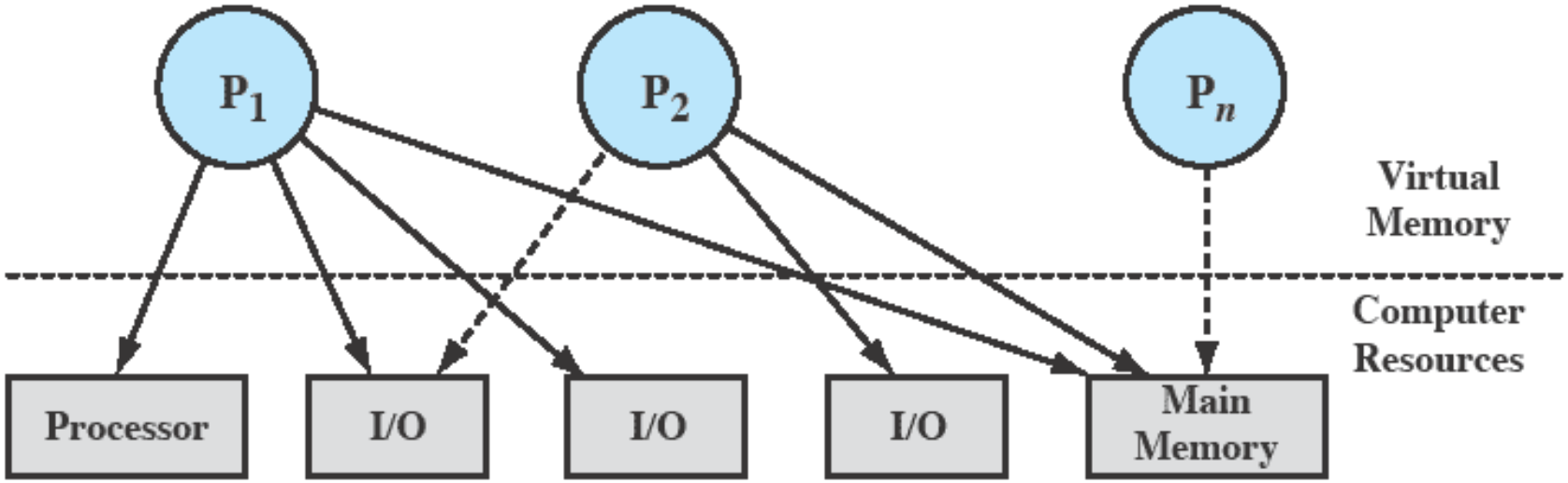


Figure 3.10 Processes and Resources (resource allocation at one snapshot in time)

Quiz: Processes states and Resources?

Control Tables in OS

- **Process Control Table:** is used to manage processes and their related information
- **Storage Control Table:** is used to manage primary (main) storage and secondary (auxiliary) storage. Its main contents include: allocation information for primary storage, allocation information for secondary storage, storage protection and partition sharing information, and virtual storage management information.
- **I/O Control Table**is used to manage the I/O devices and channels of a computer system. Its main contents include: whether I/O devices and channels are available, allocation information of I/O devices and channels, status and progress of I/O operations, and the main memory area where I/O operations transmit data.
- **File Control Table:** is used to manage files, mainly including information about opened files, the location information of files in main and secondary storage, the status of opened files, and other attribute information.

OS Control Tables

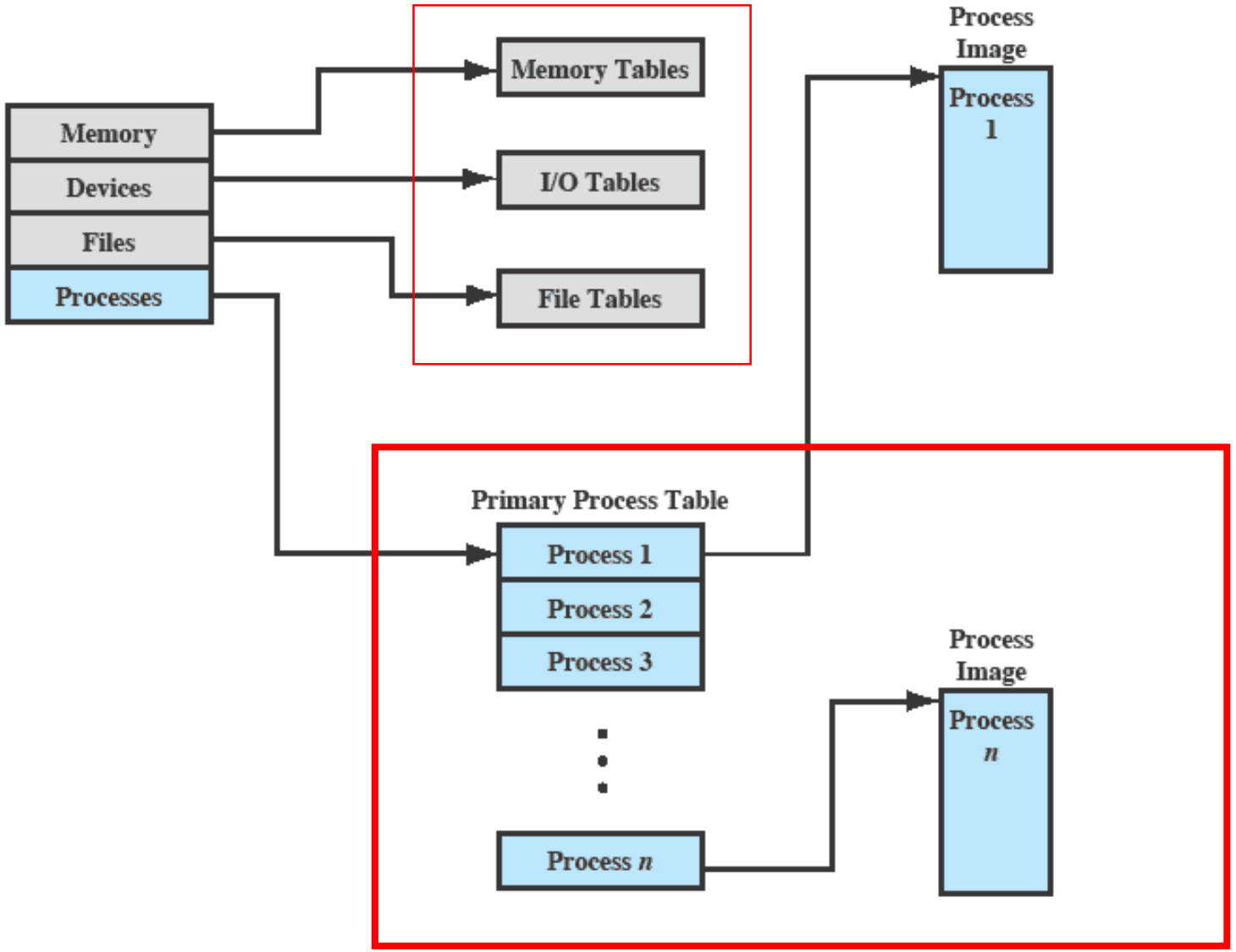


Figure 3.11 General Structure of Operating System Control Tables

西南交通大学 Southwest Jiaotong University

Process Control Structures

- To manage and control a process the OS must know:
  - where the process is located
  - the attributes of the process that are necessary for its management

- Process table（进程表）
  - Must be maintained to manage processes
  - There must be some reference to memory, I/O, and files, directly or indirectly
  - The tables themselves must be accessible by the OS and therefore are subject to memory management

Process Context （进程上下文）

- **Process Context:** In the operating system, the physical entity of a process and the environment that supports process execution are collectively referred to as the process context

  - **User level context:** A process address space composed of program blocks of user processes, user data blocks (including shared data blocks), and user stacks.
  - **System level context:** a process address space composed of process identification information, on-site information, control information, process environment blocks, and system stack.
  - **Register context:** composed of program status word registers, various control registers, address registers, general registers, user stack pointers, etc.

Process Image（进程映像）

- What is the physical manifestation (物理表示) of a process?
  - A process must include a program or set of programs to be executed
  - A process will consist of at least sufficient memory to hold the programs and data of that process
  - The execution of a program typically involves a stack that is used to keep track of procedure calls and parameter passing between procedures
  - Each process has associated with it a number of attributes that are used by the OS for process control
  - process image includes program, data, stack（堆栈）of system / users, and attributes（属性）, PCB
  - Process image location will depend on the memory management scheme being used

Process Image（进程映像）

- What is the physical manifestation (物理表示) of a process?
  - A process must include a program or set of programs to be executed
  - A process will consist of at least sufficient memory to hold the programs and data of that process
  - The execution of a program typically involves a stack that is used to keep track of procedure calls and parameter passing between procedures
  - Each process has associated with it a number of attributes that are used by the OS for process control
  - process image includes program, data, stack（堆栈）of system / users, and attributes（属性）, PCB
  - Process image location will depend on the memory management scheme being used

Typical Elements of a Process Image

User Data

The modifiable part of the user space. May include program data, a user stack area, and programs that may be modified.

User Program

The program to be executed.

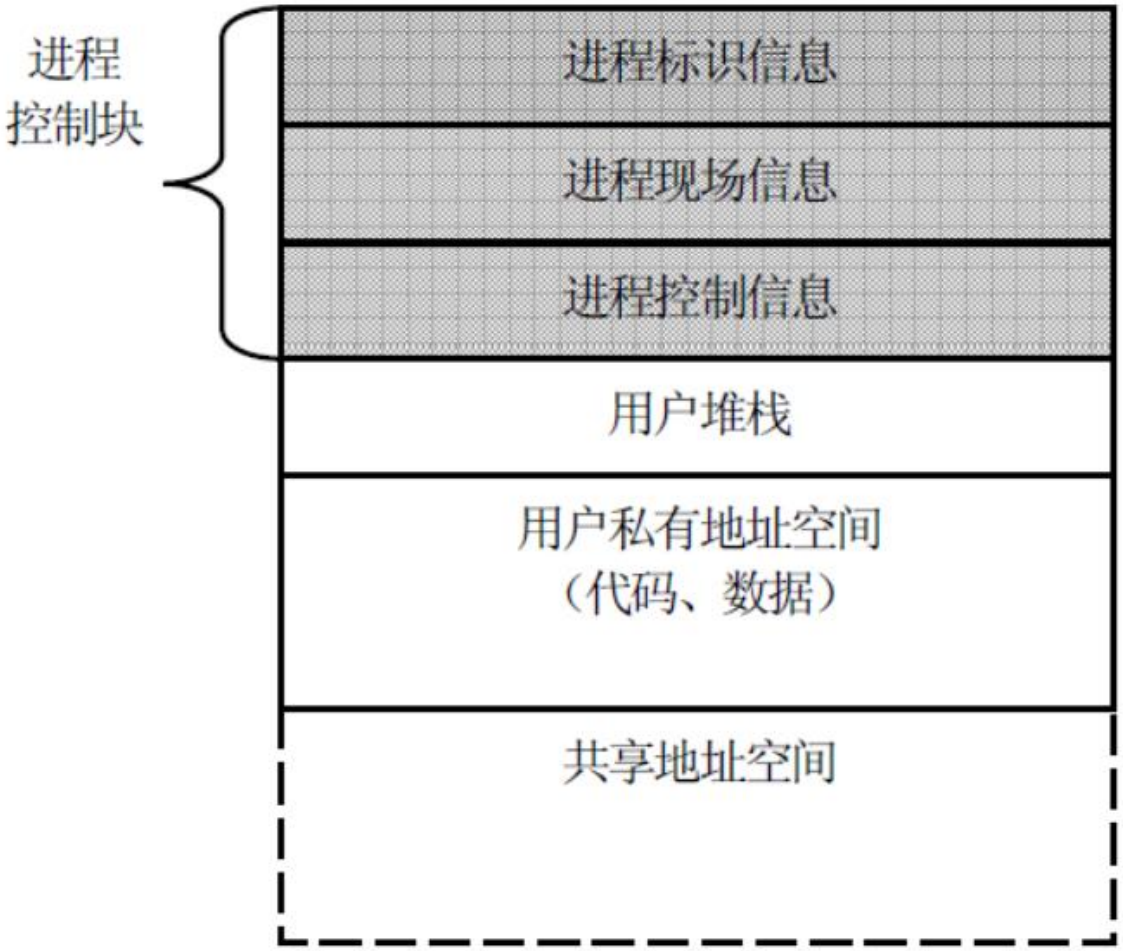
Stack

Each process has one or more last-in-first-out (LIFO) stacks associated with it. A stack is used to store parameters and calling addresses for procedure and system calls.

Process Control Block

Data needed by the OS to control the process (see Table 3.5).

Process in Virtual Memory

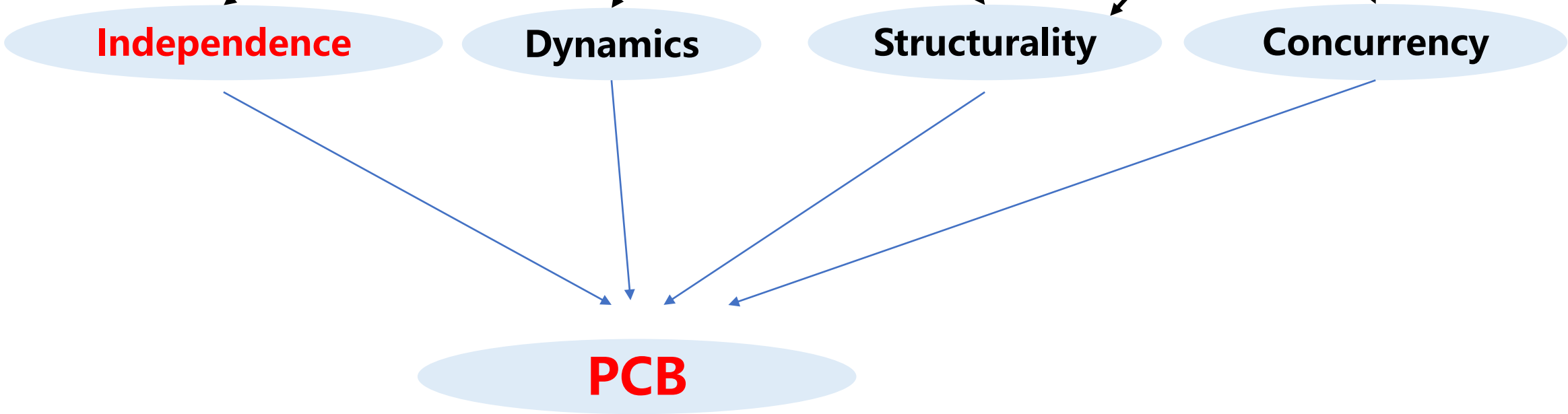


Process Control Block（进程控制块）

- The most important data structure in an OS
  - contains all of the information about a process that is needed by the OS
  - blocks are read and/or modified by every module in the OS
  - defines the state of the OS
- Difficulty is not access, but protection
  - a bug in a single routine could damage process control blocks, which could destroy the system's ability to manage the affected processes
  - a design change in the structure or semantics of the process control block could affect a number of modules in the OS
- Process attributes
  - Process identification
  - Processor state information
  - Process control information

Process Properties

- **Definition of a process: A process is one-time execution process of a concurrently executable program with independent functionality regarding a certain dataset, also the basic unit for resource allocation and protection in operating systems. (1978 National Conference on Operating Systems)**



Process Identification（进程标识符）

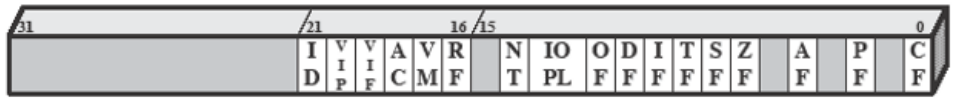
- Each process is assigned a unique numeric identifier
  - otherwise there must be a mapping that allows the OS to locate the appropriate tables based on the process identifier
- Many of the tables controlled by the OS may use process identifiers to cross-reference process tables
- Memory tables may be organized to provide a map of main memory with an indication of which process is assigned to each region
  - similar references will appear in I/O and file tables
- When processes communicate with one another, the process identifier informs the OS of the destination of a particular communication
- When processes are allowed to create other processes, identifiers indicate the parent and descendants of each process（父亲和后代进程）

Processor Status

- **Processor Status**: also known as **operating mode of the processor**. How does the central processing unit know whether it is currently running an operating system or a regular user on it? This will depend on the flags of the processor state. Set the **processor to different states based on the usage permissions of resources and machine instructions** during the execution of different programs.
- **Management Status**: Processor can execute all instructions, access all resources and have the ability to change processor status.
- **User Status**: Processor can only execute non privileged instructions.

Processor State Information （处理器状态）

- Consists of the contents of processor registers
  - user-visible registers（用户可见寄存器）
  - control and status registers
  - stack pointers（栈指针）
- All processor design include program status word (PSW) 程序状态字
  - contains condition codes （条件码） plus other status information
  - EFLAGS register is an example of a PSW used by any OS running on an x86 processor



ID = Identification flag
VIP = Virtual interrupt pending
VIF = Virtual interrupt flag
AC = Alignment check
VM = Virtual 8086 mode
RF = Resume flag
NT = Nested task flag
IOPL = I/O privilege level
OF = Overflow flag

DF = Direction flag
IF = Interrupt enable flag
TF = Trap flag
SF = Sign flag
ZF = Zero flag
AF = Auxiliary carry flag
PF = Parity flag
CF = Carry flag

Figure 3.12 Pentium II EFLAGS Register

Process Attributes

Process Identification

Identifiers

Numeric identifiers that may be stored with the process control block include

- Identifier of this process
- Identifier of the process that created this process (parent process)
- User identifier

Processor State Information

User-Visible Registers

A user-visible register is one that may be referenced by means of the machine language that the processor executes while in user mode. Typically, there are from 8 to 32 of these registers, although some RISC implementations have over 100.

Control and Status Registers

These are a variety of processor registers that are employed to control the operation of the processor. These include

- **Program counter:** Contains the address of the next instruction to be fetched
- **Condition codes:** Result of the most recent arithmetic or logical operation (e.g., sign, zero, carry, equal, overflow)
- **Status information:** Includes interrupt enabled/disabled flags, execution mode

Stack Pointers

Each process has one or more last-in-first-out (LIFO) system stacks associated with it. A stack is used to store parameters and calling addresses for procedure and system calls. The stack pointer points to the top of the stack.

Process Control Information

- The additional information needed by the OS to control and coordinate the various active processes
- Take scheduling for example

Process Control Information

Scheduling and State Information

This is information that is needed by the operating system to perform its scheduling function. Typical items of information:

- **Process state**: Defines the readiness of the process to be scheduled for execution (e.g., running, ready, waiting, halted).
- **Priority:** One or more fields may be used to describe the scheduling priority of the process. In some systems, several values are required (e.g., default, current, highest-allowable)
- **Scheduling-related information:** This will depend on the scheduling algorithm used. Examples are the amount of time that the process has been waiting and the amount of time that the process executed the last time it was running.
- **Event:** Identity of event the process is awaiting before it can be resumed.

Data Structuring

A process may be linked to other process in a queue, ring, or some other structure. For example, all processes in a waiting state for a particular priority level may be linked in a queue. A process may exhibit a parent-child (creator-created) relationship with another process. The process control block may contain pointers to other processes to support these structures.

Interprocess Communication

Various flags, signals, and messages may be associated with communication between two independent processes. Some or all of this information may be maintained in the process control block.

Process Privileges

Processes are granted privileges in terms of the memory that may be accessed and the types of instructions that may be executed. In addition, privileges may apply to the use of system utilities and services.

Memory Management

This section may include pointers to segment and/or page tables that describe the virtual memory assigned to this process.

Resource Ownership and Utilization

Resources controlled by the process may be indicated, such as opened files. A history of utilization of the processor or other resources may also be included; this information may be needed by the scheduler.

2. Process Description and Control

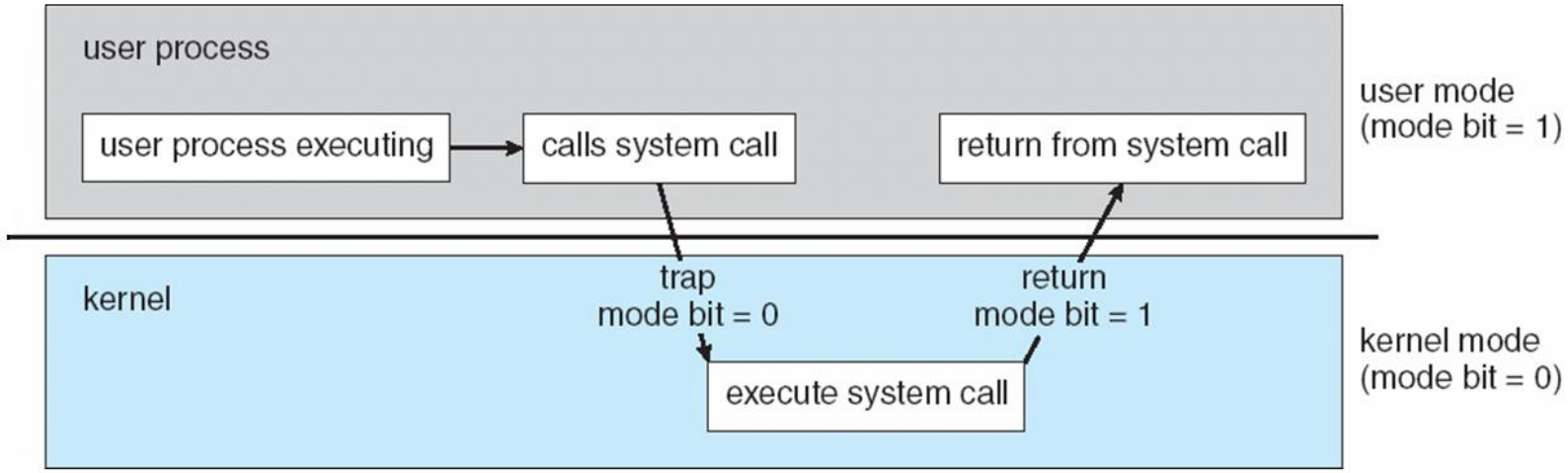
- Process concept
- Process description
- **Process execution**

Process Control

- **Modes of execution**
- Process creation
- Process switching

Modes of Execution

- User Mode
  - less-privileged mode
  - user programs typically execute in this mode
  - PSW-cpl (2bits)
- System Mode
  - more-privileged mode
  - also referred to as control mode or kernel mode
  - kernel of the operating system



西南交通大學
Southwest Jiaotong University

Typical Functions of an Operating System Kernel

Process Management

- Process creation and termination
- Process scheduling and dispatching
- Process switching
- Process synchronization and support for interprocess communication
- Management of process control blocks

Memory Management

- Allocation of address space to processes
- Swapping
- Page and segment management

I/O Management

- Buffer management
- Allocation of I/O channels and devices to processes

Support Functions

- Interrupt handling
- Accounting
- Monitoring

西南交通大學
Southwest Jiaotong University

Process Control

- Modes of execution
- **Process creation**
- Process switching

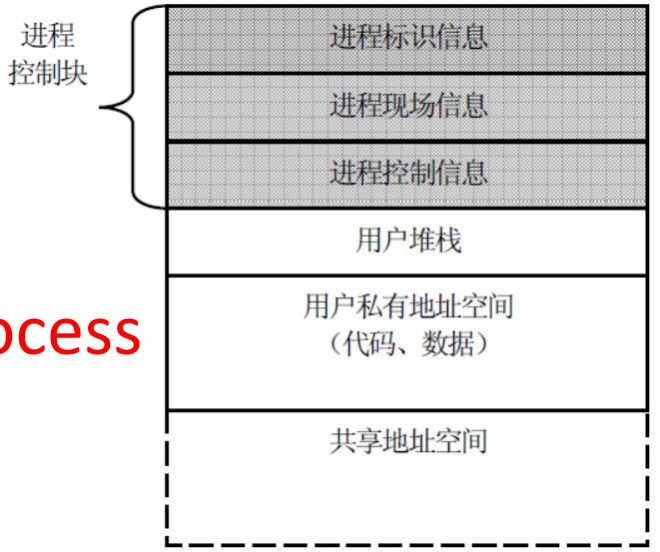
Reasons for Process Creation

- UNIX: fork
- WINDOWS: CreateProcess

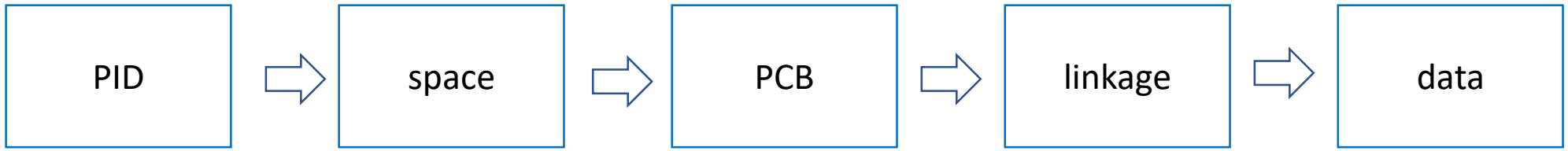
New batch job	The OS is provided with a batch job control stream, usually on tape or disk. When the OS is prepared to take on new work, it will read the next sequence of job control commands.
Interactive logon	A user at a terminal logs on to the system.
Created by OS to provide a service	The OS can create a process to perform a function on behalf of a user program, without the user having to wait (e.g., a process to control printing).
Spawned by existing process	For purposes of modularity or to exploit parallelism, a user program can dictate the creation of a number of processes.

Parent and child processes have their own distinct address spaces

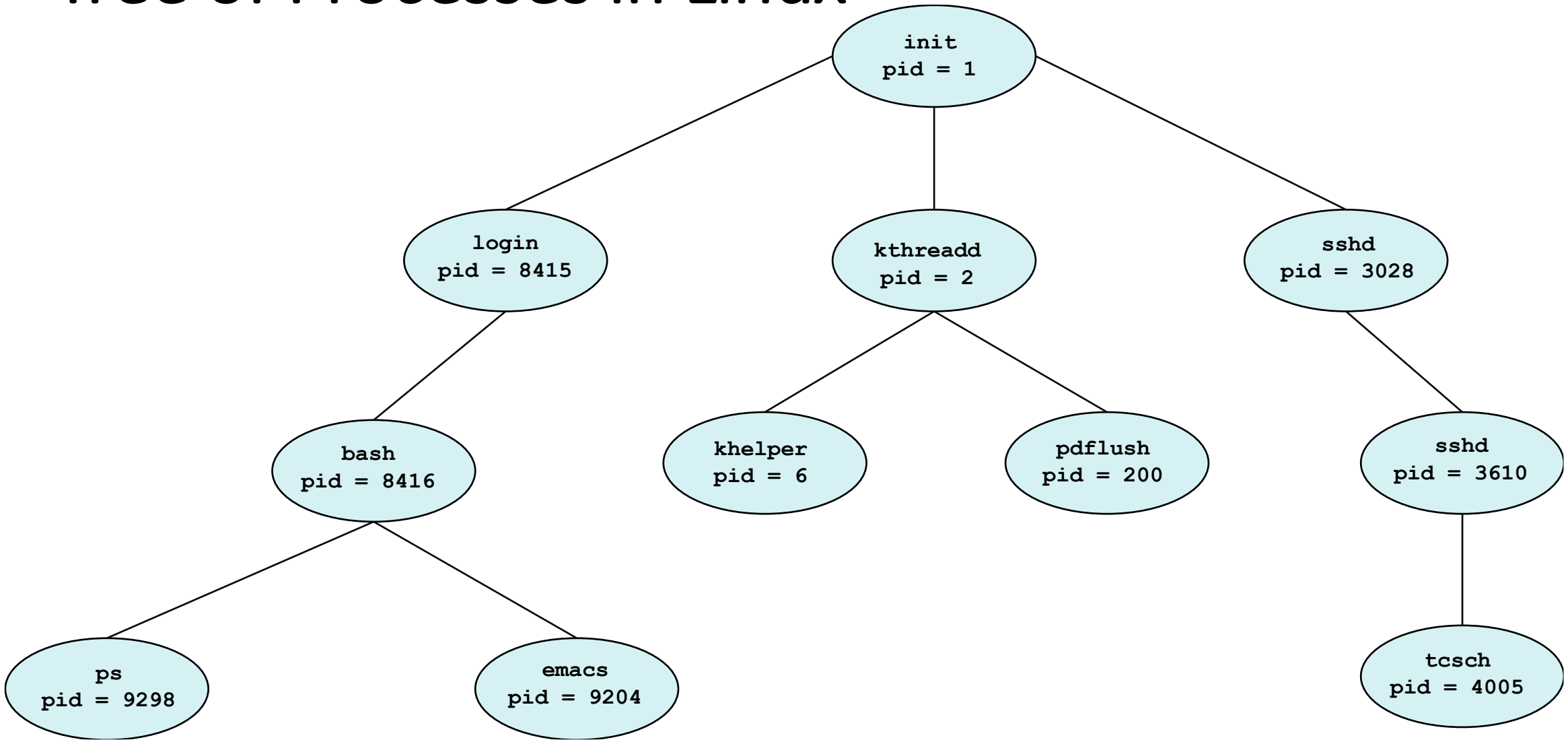
Step of Process Creation



- Once the OS decides to create a new process
  - ① assigns a unique process identifier to the new process
  - ② allocates space for the process
  - ③ initializes the PCB
  - ④ sets the appropriate linkages
  - ⑤ creates or expands other data structures



Tree of Processes in Linux



Process Creation

- Parent process create children processes, which, in turn create other processes, forming a tree of processes

- Generally, process identified and managed via a process identifier (pid)

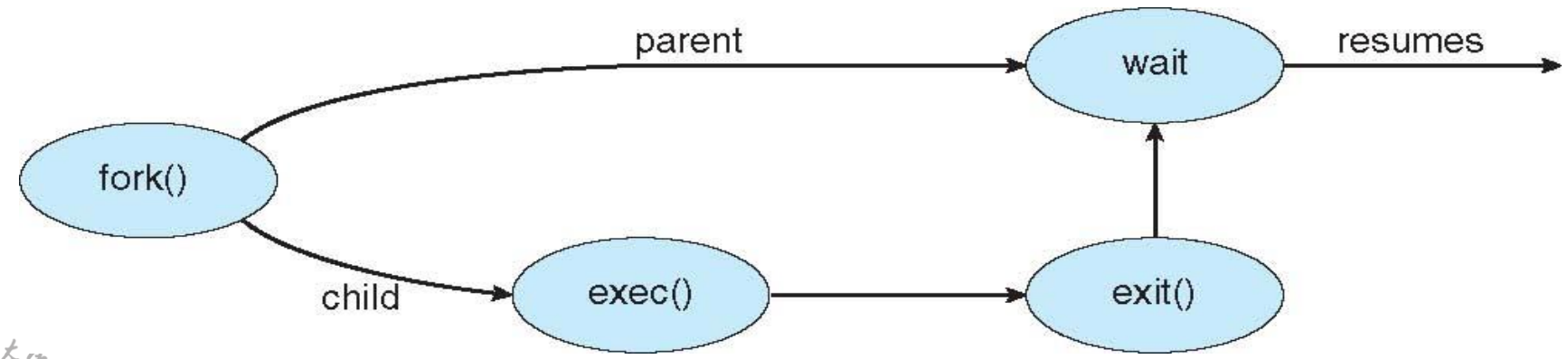
- Resource sharing options
  - Parent and children share all resources
  - Children share subset of parent's resources
  - Parent and child share no resources

- Execution options
  - Parent and children execute concurrently
  - Parent waits until children terminate

Process Creation

- Address space
  - Child duplicate of parent
  - Child has a program loaded into it

- UNIX examples
  - fork() system call creates new process
  - exec() system call used after a fork() to replace the process' memory space with a new program

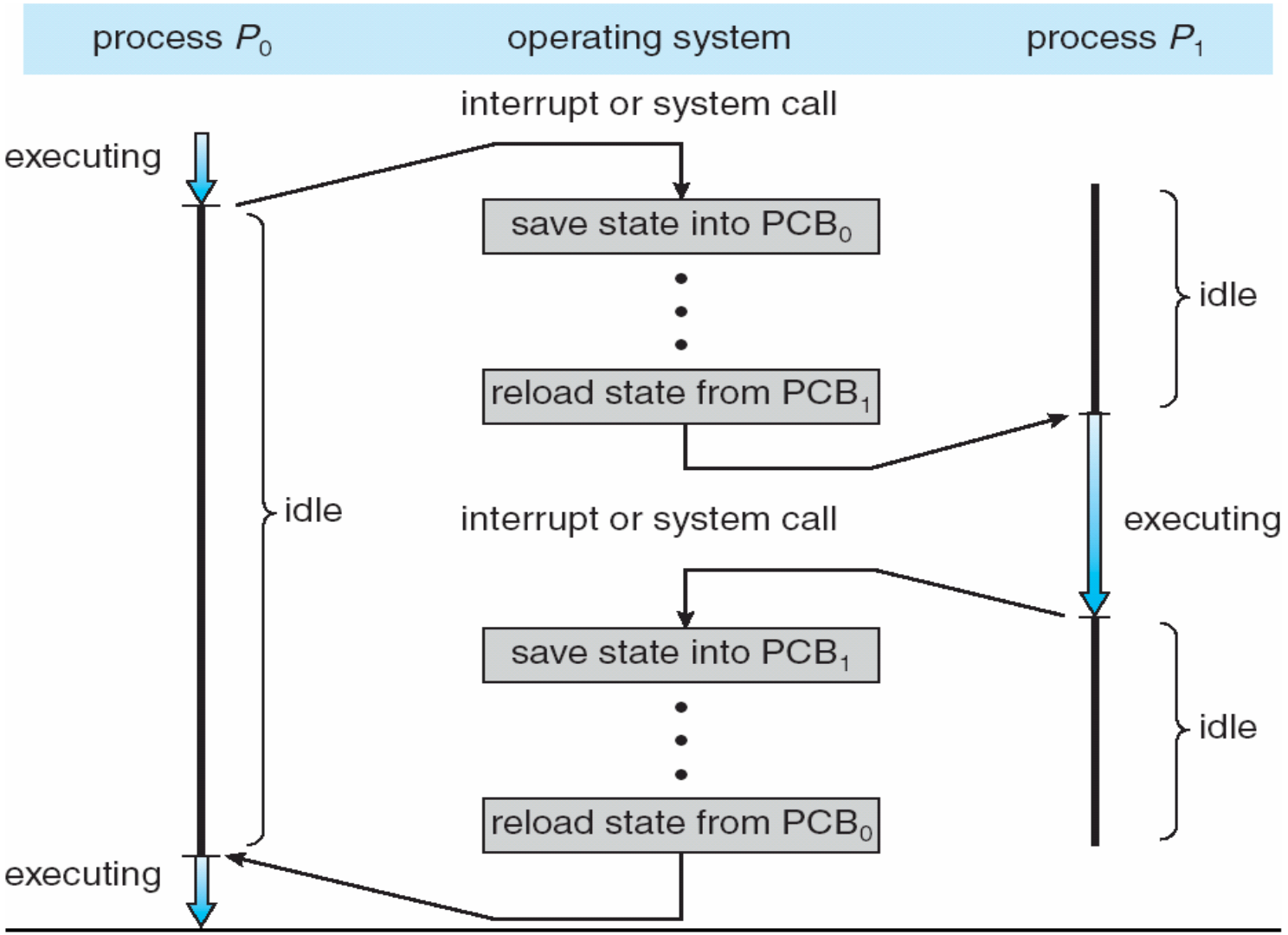


西南交通大学
Southwest Jiaotong University

Process Control

- Modes of execution
- Process creation
- **Process switching**

Process Switching



Change of Process State

- The steps in a full process switch are:
  1. save the context of the processor
  2. update the process control block of the process currently in the Running state
  3. move the process control block of this process to the appropriate queue
  4. select another process for execution
  5. update the process control block of the process selected
  6. update memory management data structures
  7. restore the context of the processor to that which existed at the time the selected process was last switched out

context

state

queue

another process

PCB

memory

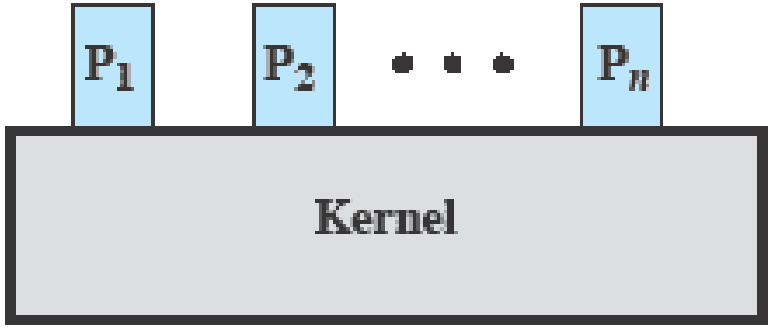
context

Process Switching

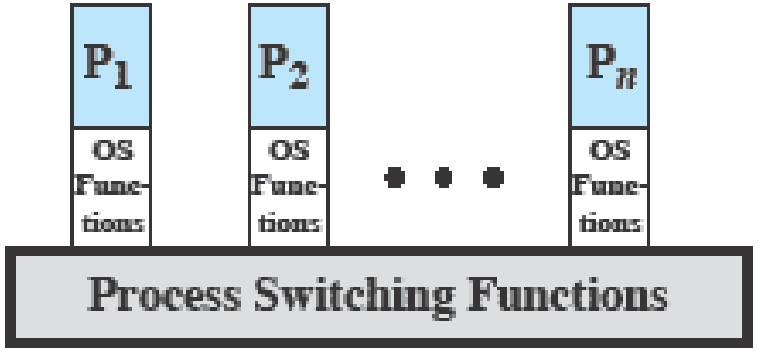
- A process switch may occur any time that the OS has gained control from the currently running process.
  - Possible events giving OS control are: Interrupt, Trap and Supervisor call (系统调用)

- Interrupt (中断)
  - Due to some external event, which is independent of the currently running process
    - ✓ clock interrupt
    - ✓ I/O interrupt
    - ✓ memory fault
- Trap （陷阱）
  - An error or exception condition generated within the currently running process
  - OS determines if the condition is fatal
    - ✓ moved to the Exit state and a process switch occurs
    - ✓ action will depend on the nature of the error

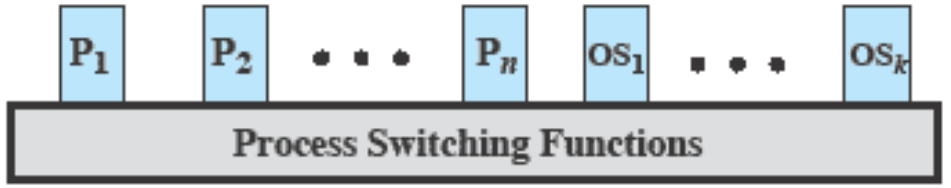
Execution of the OS



(a) Separate kernel



(b) OS functions execute within user processes



(c) OS functions execute as separate processes

Figure 3.15 Relationship Between Operating System and User Processes

西南交通大學
Southwest Jiaotong University

Process Switching

- Nonprocess kernel
  - Kernel is outside of any process
  - OS has its own region of memory and system stack
  - Concept of process applies only to user program.
- Execution within user processes
  - Small computers
  - OS is viewed as a collection of routines（例程）
  - When an interrupt occurs, mode switching passes control to OS, and performs mode switch in the same process.
- Process-based OS
  - Multiprocessor environment
  - OS is viewed as a collection of system processes
  - Clean interfaces between modules

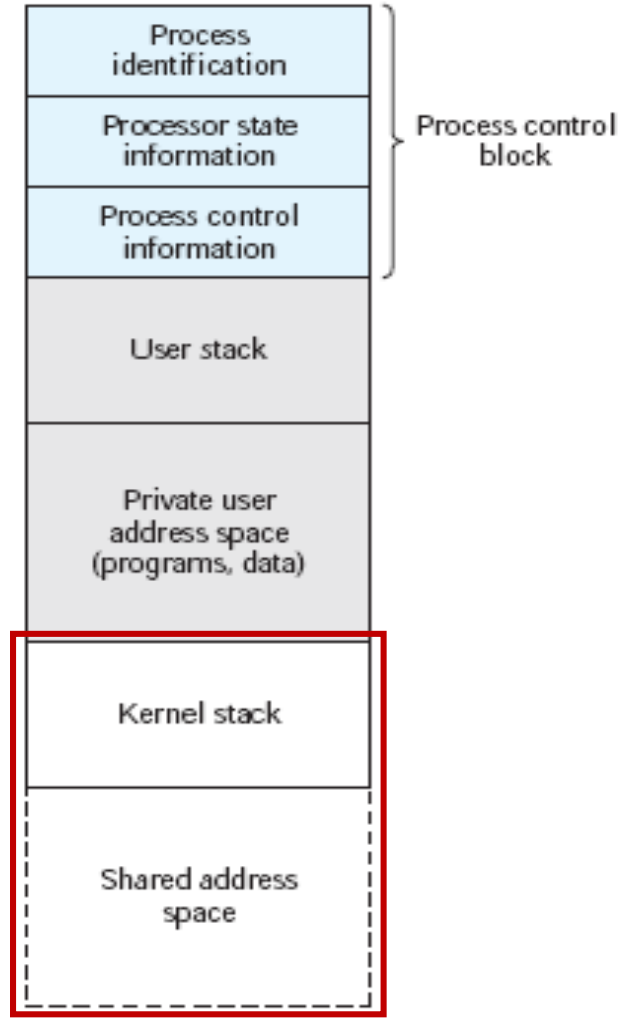
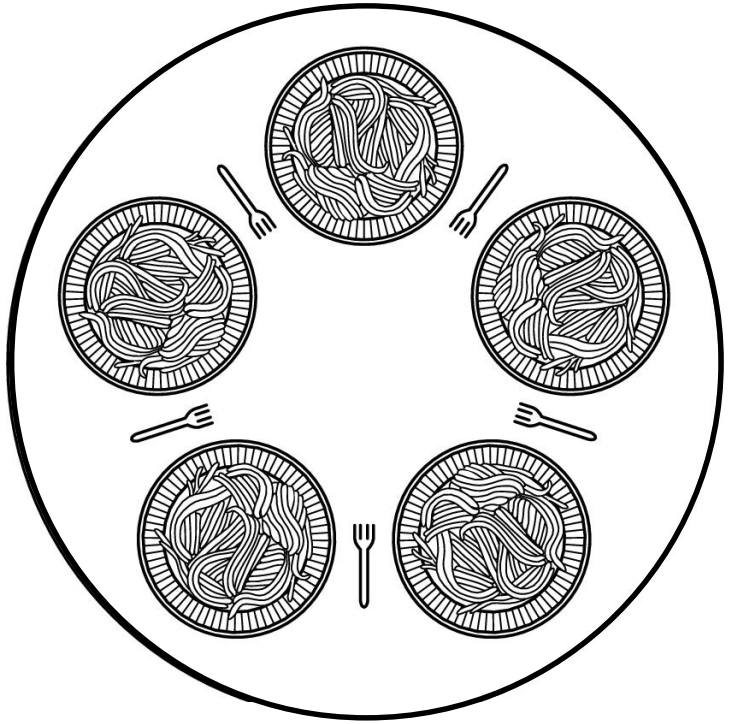


Figure 3.16 Process Image: Operating System Executes within User Space

Process Switching

- **Dining Philosophers Problem（Inter-Process Communication）**



Summary

- Process VS program
- Process states model, eg 2,5,7
- Process description
  - Process image: program, data, stack, PCB
  - PCB: identification, state info and control info
- Process execution
  - Modes of execution: kernel and user
  - Creation
  - Change of process state, 7 steps

Exercise

1. Definition and Properties of Process
2. What is PSW? What is its main function?
3. Discuss the advantages of multiple CPU states over two CPU states.
4. If there is a set of assignments J1,..., Jn, their execution times are S1,..., Sn in sequence. If these tasks arrive at the system simultaneously and are executed in a single pass manner on a single CPU processor. Try to find a job scheduling algorithm that minimizes the average job turnover time.